

**KHOA CÔNG NGHỆ THÔNG TIN 1**



**BÁO CÁO THỰC TẬP CƠ SỞ**

**ĐỀ TÀI: Xây dựng nền tảng hỗ trợ làm việc nhóm từ xa tương tác trực quan với bảng trắng EvoDraw**

**Giảng viên hướng dẫn: TS.ĐỖ THỊ LIÊN**

**Sinh viên thực hiện : ĐÀO VĂN KHANG VŨ MINH SÁNG**

**Mã sinh viên : B23DCVT218 B23DCCE082**

**Hà Nội - 2026**

## **LỜI CẢM ƠN {#lời-cảm-ơn}**

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới TS. Đỗ Thị Liên người đã hướng dẫn chúng em thực hiện đề tài này trong học phần Thực tập cơ sở. Trong suốt quá trình thực hiện đề tài, cô không chỉ định hướng cho em một đề tài nghiên cứu có ý nghĩa thực tiễn, mà còn dành nhiều thời gian, tâm huyết để tận tình hướng dẫn, đồng hành và tạo điều kiện thuận lợi cho chúng em hoàn thành đề tài này.

Thông qua đề tài này, chúng em đã vận dụng những kiến thức lý thuyết đã học về phân tích thiết kế hướng đối tượng (OOAD), mô hình hóa UML, kiến trúc Client-Server và các công nghệ web hiện đại để phân tích, thiết kế và xây dựng một hệ thống cộng tác thời gian thực sát với thực tế nhất. Đề tài tập trung giải quyết các bài toán cốt lõi như quản lý phòng họp, đồng bộ bảng trắng đa người dùng, chia sẻ màn hình kết hợp ghi chú trực tiếp, gọi thoại qua SFU (LiveKit) và truyền tải tin nhắn tức thời.

Chúng em đã nỗ lực hết sức, làm việc nghiêm túc và vận dụng những kiến thức đã học để hoàn thành đề tài. Nhưng do thời gian thực hiện có hạn và kinh nghiệm thực tiễn chưa nhiều, bài báo cáo chắc chắn không tránh khỏi những thiếu sót.

Chúng em rất mong nhận được sự đánh giá, góp ý của cô để bài làm được hoàn thiện hơn, đồng thời giúp chúng em đúc kết thêm những kinh nghiệm quý báu cho công việc sau này.

Chúng em xin chân thành cảm ơn!

# MỤC LỤC {#mục-lục}

---

## LỜI CẢM ƠN 2

## MỤC LỤC 3

## DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT 5

## DANH MỤC CÁC BẢNG 8

## DANH MỤC CÁC HÌNH VẼ 10

## ĐẶT VẤN ĐỀ 12

## CHƯƠNG 1 - SỰ TƯƠNG TÁC TRỰC QUAN TRONG LÀM VIỆC NHÓM TỪ XA 13

1.1. Xác định yêu cầu cho nền tảng hỗ trợ làm việc nhóm từ xa tương tác trực quan với bảng trắng. 13

1.2 Khảo sát những nghiên cứu liên quan 13

1.3 Mô hình nghiệp vụ bằng ngôn ngữ tự nhiên 15

1.4. Xây dựng sơ đồ usecase 22

1.5 Thiết kế tương tác cho sản phẩm 25

## CHƯƠNG 2 : NGHIÊN CỨU PHƯƠNG PHÁP TIẾP CẬN VÀ GIẢI QUYẾT VẤN ĐỀ 27

2.1. Mô hình tổng quát hệ thống 27

2.1.1. Tổng quan hệ thống 27

2.1.2. Sơ đồ kiến trúc tổng quát 28

2.1.3. Các luồng luân chuyển dữ liệu chính 29

2.1.4. Cơ chế đồng bộ và chiến lược giải quyết xung đột 29

2.2 Phương pháp xây dựng phần mềm 30

2.3 Mô hình phát triển phần mềm 31

2.4 Kiến trúc phần mềm được áp dụng trong triển khai lập trình hệ thống 33

2.5. Lựa chọn công nghệ phù hợp để triển khai xây dựng hệ thống 34

2.5.1. Công nghệ cho Module Desktop App 34

2.5.2. Công nghệ cho Module Web App 35

2.5.3. Công nghệ cho Module Backend Service 35

2.6. Kết luận Chương 2 37

## **CHƯƠNG 3 : Phân tích, thiết kế và thực nghiệm hệ thống 39**

3.1 Phân tích thiết kế hệ thống: 39

3.1.1. Các kịch bản (Scenarios) cho các Use Case 39

a. Kịch bản tạo phòng mới 39

b. Kịch bản tham gia phòng 39

c. Kịch bản vẽ tự do trên bảng trắng 40

d. Kịch bản chèn ảnh vào bảng trắng 40

e. Kịch bản chia sẻ màn hình 41

f. Kịch bản gọi thoại (Voice Chat) 42

g. Kịch bản gửi tin nhắn chat 42

h. Kịch bản xuất và nhập bảng trắng 43

i. Kịch bản rời phòng 43

3.1.2. Trích các lớp phân tích 44

3.1.3. Phân tích chi tiết từng module 46

a. Chức năng Tạo phòng mới 46

b. Chức năng Tham gia phòng 50

c. Chức năng Vẽ tự do trên bảng trắng 53

d. Chức năng Chèn ảnh vào bảng trắng 57

e. Chức năng Chia sẻ màn hình 61

f. Chức năng Gọi thoại (Voice Chat) 64

g. Chức năng Gửi tin nhắn chat 67

h. Chức năng Nhập và xuất file bảng trắng 68

i. Chức năng rời phòng 72

3.1.4. Thiết kế chi tiết hệ thống 76

a. Hoàn thiện cấu trúc các thành phần thiết kế 76

b. Thiết kế chi tiết các mô-đun (Dạng bảng) 78

### 3.2. Thiết kế cơ sở dữ liệu 83

#### 3.2.1. Mô hình dữ liệu (Document Model) 83

#### 3.2.2 Thiết kế vật lý (Physical Design) 84

### 3.3 Thực nghiệm (nếu có) : 87

## KẾT LUẬN, KIẾN NGHỊ 88

## TÀI LIỆU THAM KHẢO 89

## PHỤ LỤC CÀI ĐẶT VÀ TRIỂN KHAI : 90

---

# DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

## {#danh-muc-cac-ky-hieu-va-chu-viet-tat}

---

| Ký hiệu, viết tắt | Tiếng Anh                              | Tiếng Việt                            |
|-------------------|----------------------------------------|---------------------------------------|
| API               | Application Programming Interface      | Giao diện lập trình ứng dụng          |
| REST              | Representational State Transfer        | Chuyển đổi trạng thái đại diện        |
| P2P               | Peer-to-Peer                           | Mạng ngang hàng                       |
| WebRTC            | Web Real-Time Communication            | Truyền thông thời gian thực trên Web  |
| SDP               | Session Description Protocol           | Giao thức mô tả phiên                 |
| ICE               | Interactive Connectivity Establishment | Thiết lập kết nối tương tác           |
| WebSocket         | —                                      | Giao thức truyền thông hai chiều      |
| OOAD              | Object-Oriented Analysis and Design    | Phân tích và thiết kế hướng đối tượng |
| UML               | Unified Modeling Language              | Ngôn ngữ mô hình hóa thống nhất       |
| BCE               | Boundary – Control – Entity            | Biên – Điều khiển – Thực thể          |
| LWW               | Last-Writer-Wins                       | Bản ghi cuối cùng thắng               |
| TTL               | Time-To-Live                           | Thời gian tồn tại                     |
| MVP               | Minimum Viable Product                 | Sản phẩm khả dụng tối thiểu           |
| NSD               | —                                      | Người sử dụng                         |
| CSDL              | —                                      | Cơ sở dữ liệu                         |
| CDN               | Content Delivery Network               | Mạng phân phối nội dung               |
| UUID              | Universally Unique Identifier          | Định danh duy nhất toàn cầu           |

| Ký hiệu, viết tắt | Tiếng Anh                             | Tiếng Việt                          |
|-------------------|---------------------------------------|-------------------------------------|
| MIME              | Multipurpose Internet Mail Extensions | Chuẩn mở rộng nội dung Internet     |
| BSON              | Binary JSON                           | Định dạng JSON nhị phân             |
| JSON              | JavaScript Object Notation            | Định dạng đối tượng JavaScript      |
| PNG               | Portable Network Graphics             | Đồ họa mạng di động                 |
| PDF               | Portable Document Format              | Định dạng tài liệu di động          |
| GIF               | Graphics Interchange Format           | Định dạng trao đổi hình ảnh         |
| SVG               | Scalable Vector Graphics              | Đồ họa vector có thể thu phóng      |
| URL               | Uniform Resource Locator              | Trình định vị tài nguyên thống nhất |
| RAM               | Random Access Memory                  | Bộ nhớ truy cập ngẫu nhiên          |
| NoSQL             | Not Only SQL                          | Cơ sở dữ liệu phi quan hệ           |
| ODM               | Object-Document Mapper                | Ánh xạ đối tượng - tài liệu         |
| DOM               | Document Object Model                 | Mô hình đối tượng tài liệu          |
| PIN               | Personal Identification Number        | Mã số nhận dạng cá nhân             |
| Base64            | —                                     | Mã hóa nhị phân sang văn bản        |

## DANH MỤC CÁC BẢNG {#danh-muc-cac-bang}

Bảng 1.1: Khảo sát ưu, nhược điểm các nền tảng hỗ trợ làm việc nhóm từ xa 14

Bảng 1.2: Yêu cầu chức năng của hệ thống EvoDraw 17

Bảng 1.3: Yêu cầu phi chức năng của hệ thống EvoDraw 18

Bảng 2.1: Tổng hợp công nghệ sử dụng cho từng module 37

Bảng 3.1: Mô-đun SyncController.broadcastOperation 78

Bảng 3.2: Mô-đun SyncController.applyRemoteOperation 79

Bảng 3.3: Mô-đun MediaController.uploadFile 79

Bảng 3.4: Mô-đun RoomController.createRoom 80

Bảng 3.5: Mô-đun RoomController.joinRoom 80

Bảng 3.6: Mô-đun RoomController.leaveRoom 80

Bảng 3.7: Mô-đun StreamController.startSharing 81

Bảng 3.8: Mô-đun StreamController.toggleVoice 81

Bảng 3.9: Mô-đun ChatController.sendMessage 82

Bảng 3.10: Mô-đun DocumentController.importFile 82

Bảng 3.11: Mô-đun DocumentController.exportFile 83

---

## **DANH MỤC CÁC HÌNH VẼ** {#danh-mục-các-hình-vẽ}

---

Hình 1.1: Sơ đồ Use Case tổng quan của hệ thống EvoDraw 21

Hình 1.2: Sơ đồ Use Case phân rã — Vẽ lên bảng trắng 22

Hình 1.3: Sơ đồ Use Case phân rã — Chia sẻ màn hình 23

Hình 1.4: Sơ đồ Use Case phân rã — Quản lý dữ liệu canvas 23

Hình 1.6: Thiết kế giao diện Trang chủ (Tạo / Tham gia phòng) 24

Hình 1.7: Thiết kế giao diện Phòng làm việc (bảng trắng + thanh công cụ) 24

Hình 1.8: Thiết kế giao diện Chia sẻ màn hình 25

Hình 1.9: Thiết kế giao diện Cài đặt / Tùy chỉnh phòng 25

Hình 2.1: Sơ đồ tổng quát hệ thống EvoDraw 27

Hình 3.1: Biểu đồ lớp thực thể tổng quan của hệ thống EvoDraw 45

Hình 3.2: Biểu đồ các lớp phân tích — chức năng Tạo phòng mới 46

Hình 3.3: Biểu đồ tuần tự — chức năng Tạo phòng mới 48

Hình 3.4: Biểu đồ các lớp phân tích — chức năng Tham gia phòng 50

Hình 3.5: Biểu đồ tuần tự — chức năng Tham gia phòng 51

Hình 3.6: Biểu đồ các lớp phân tích — chức năng Vẽ tự do trên bảng trắng 53

Hình 3.7: Biểu đồ tuần tự — chức năng Vẽ tự do trên bảng trắng 55

Hình 3.8: Biểu đồ các lớp phân tích — chức năng Chèn ảnh vào bảng trắng 57

Hình 3.9: Biểu đồ tuần tự — chức năng Chèn ảnh vào bảng trắng 59

Hình 3.10: Biểu đồ các lớp phân tích — chức năng Chia sẻ màn hình 60

Hình 3.11: Biểu đồ tuần tự — chức năng Chia sẻ màn hình 62

Hình 3.12: Biểu đồ các lớp phân tích — chức năng Gọi thoại (Voice Chat) 63

Hình 3.13: Biểu đồ tuần tự — chức năng Gọi thoại (Voice Chat) 64

Hình 3.14: Biểu đồ các lớp phân tích — chức năng Gửi tin nhắn chat 65

Hình 3.15: Biểu đồ tuần tự — chức năng Gửi tin nhắn chat 66

Hình 3.16: Biểu đồ các lớp phân tích — chức năng Nhập và xuất file bảng trắng 68

Hình 3.17: Biểu đồ tuần tự — chức năng Nhập file bảng trắng 69

Hình 3.18: Biểu đồ tuần tự — chức năng Xuất file bảng trắng 70

---

## ĐẶT VẤN ĐỀ {#đặt-vấn-đề}

---

Trong bối cảnh thế giới việc làm đang trải qua cuộc cách mạng mạnh mẽ sau đại dịch, theo báo cáo của McKinsey, mô hình làm việc kết hợp giữa làm việc tại văn phòng và làm việc tại nhà (hybrid working) đã trở thành ‘tiêu chuẩn vàng’ với hơn 66% doanh nghiệp áp dụng. Thế nhưng, một thực tế đáng báo động là hiệu suất làm việc nhóm từ xa đang bị kìm hãm bởi sự hạn chế của các công cụ chia sẻ màn hình truyền thống - nơi người xem chỉ đóng vai trò thụ động.

Việc thiếu khả năng tương tác trực tiếp trên nội dung đang thảo luận khiến các cuộc họp trở nên rời rạc, khó nắm bắt và thiếu tính sáng tạo. EvoDraw ra đời để giải quyết triệt để vấn đề này bằng cách xây dựng một nền tảng làm việc nhóm tại một không gian bảng trắng kỹ thuật số (whiteboard) linh hoạt. Đây là nơi mọi thành viên đều có thể trực tiếp tương tác với bảng trắng (vẽ, ghi chú, đánh dấu,...) kết hợp với tính năng chia sẻ màn hình để mọi ý tưởng được diễn giải một cách trực quan và liền mạch nhất.

Trong báo cáo sẽ tập trung nghiên cứu xây dựng nền tảng hỗ trợ làm việc nhóm từ xa tương tác trực quan với bảng trắng. Với chương 1 là giới thiệu tổng quan về sự tương tác trực quan trong làm việc nhóm từ xa, bao gồm xác định yêu cầu của hệ thống, khảo sát một vài nền tảng tương tự, thiết kế tương tác. Chương 2 sẽ tập trung vào phương pháp tiếp cận và giải pháp mà nhóm đề xuất, bao gồm mô hình tổng quát, phương pháp xây dựng, mô hình phát triển, kiến trúc phần mềm, công nghệ phù hợp để triển khai xây dựng hệ thống. Chương 3 tập trung vào phân tích và thiết kế hệ thống, bao gồm các biểu đồ và Scenario cho các Use case, thiết kế cơ sở dữ liệu và thực nghiệm liên quan. Sau đó đến phần kết luận về kết quả đạt được và hướng phát triển trong tương lai. Cuối cùng, phần phụ lục là quá trình cài đặt, triển khai hệ thống và hình ảnh sản phẩm.

---

## CHƯƠNG 1 - SỰ TƯƠNG TÁC TRỰC QUAN TRONG LÀM VIỆC NHÓM TỪ XA {#chương-1---sự-tương-tác-trực-quan-trong-làm-việc-nhóm-từ-xa}

---

Chương này sẽ tập trung vào việc xác định yêu cầu cho nền tảng hỗ trợ làm việc nhóm từ xa tương tác trực quan với bảng trắng EvoDraw. Chúng ta sẽ bắt đầu bằng việc nắm rõ mục đích của hệ thống trong kỷ nguyên hybrid working, khảo sát các giải pháp cộng tác trực tuyến hiện có, và chi tiết hóa yêu cầu hoạt động của ứng dụng dành cho người dùng. Tiếp theo, sẽ là phần tạo hình sản phẩm thông qua thiết kế tương tác, tập trung vào trải nghiệm người dùng. Mục tiêu là xây dựng nền móng vững chắc cho quá trình phát triển, làm cơ sở cho các chương sau.

## 1.1. Xác định yêu cầu cho nền tảng hỗ trợ làm việc nhóm từ xa tương tác trực quan với bảng trắng. {#1.1.-xác-định-yêu-cầu-cho-nền-tảng-hỗ-trợ-làm-việc-nhóm-từ-xa-tương-tác-trực-quan-với-bảng-trắng.}

Đây là một nền tảng để cộng tác trực quan từ xa, cho phép người dùng chia sẻ màn hình và thực hiện các ghi chú, vẽ tương tác trực tiếp theo thời gian thực thông qua giao diện trình duyệt hoặc ứng dụng window. Tạo ra một trải nghiệm làm việc nhóm linh hoạt cho người dùng, bao gồm khả năng đồng bộ hóa nét vẽ tức thì, hỗ trợ nhiều công cụ đánh dấu đa dạng và hiển thị con trỏ chuột của các thành viên để tăng tính định hướng. Bên cạnh đó, tạo ra một môi trường giúp người trong cuộc họp có thể lưu trữ các bản vẽ ghi chú, đảm bảo bảo mật đường truyền và tối ưu hóa hiệu suất kết nối, đảm bảo hoạt động suôn sẻ và nâng cao hiệu quả sáng tạo trong làm việc tập thể.

## 1.2 Khảo sát những nghiên cứu liên quan {#1.2-khảo-sát-những-nghiên-cứu-liên-quan}

Sau khi tìm hiểu và trải nghiệm 3 nền tảng hỗ trợ làm việc nhóm từ xa nổi tiếng: Miro, Google Meet, Zoom có những ưu, nhược điểm như sau:

| Nền tảng    | Ưu điểm                                                                                                                                                                 | Nhược điểm                                                                                                                                                                                                      |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Miro        | - Bảng trắng vô hạn, cực kỳ mạnh mẽ cho tư duy hình ảnh. - Kho mẫu đồ sộ cho brainstorming, sơ đồ tư duy. - Hỗ trợ nhiều người cùng tương tác một lúc mượt mà.          | - Tập trung vào bảng trắng, không mạnh về chia sẻ màn hình ứng dụng trực tiếp. - Giao diện phức tạp, đòi hỏi thời gian làm quen cao. - Gói miễn phí bị giới hạn số lượng bảng.                                  |
| Google Meet | - Tích hợp sâu vào hệ sinh thái Google (Calendar, Drive). - Sử dụng trực tiếp trên trình duyệt, không cần cài đặt. - Tính năng bảng trắng đi kèm khá đơn giản, dễ dùng. | - Tính năng vẽ/ghi chú khi đang chia sẻ màn hình rất hạn chế. - Người xem đóng vai trò thụ động, khó có thể tương tác trực tiếp lên màn hình người khác đang chia sẻ.                                           |
| Zoom        | - Chất lượng truyền tải hình ảnh/âm thanh ổn định. - Có tính năng ghi chú cho phép người xem vẽ lên màn hình chia sẻ.                                                   | - Phải cài đặt phần mềm để sử dụng đầy đủ tính năng. - Tính năng ghi chú đôi khi gây rối mắt nếu không quản lý tốt quyền hạn. - Trải nghiệm bảng trắng không chuyên sâu bằng các nền tảng chuyên biệt như Miro. |



=> Dựa vào các ưu, nhược điểm của các nền tảng trên, EvoDraw cần đáp ứng được các yêu cầu:

- Đồng bộ hóa nét vẽ và con trỏ chuột của mọi thành viên với độ trễ cực thấp, xóa bỏ sự “rời rạc” giữa lời nói và hình ảnh.
- Cho phép vẽ ghi chú đè lên luồng màn hình đang chia sẻ thay vì ảnh tĩnh, giúp nắm bắt nội dung đang chuyển động một cách chính xác.
- Loại bỏ thủ tục đăng nhập phức tạp, cho phép tham gia nhanh qua liên kết, mã phòng để tối ưu hóa thời gian bắt đầu cuộc họp.
- Khắc phục việc mất dữ liệu khi dừng chia sẻ bằng cách cho phép lưu lại các lớp ghi chú để làm tài liệu tham chiếu sau cuộc họp.
- Hiển thị vị trí thao tác của từng người dùng cụ thể, giúp mọi thành viên luôn tập trung vào cùng một tiêu điểm thảo luận.

### **1.3 Mô hình nghiệp vụ bằng ngôn ngữ tự nhiên {#1.3-mô-hình-nghiệp-vụ-bằng-ngôn-ngữ-tự-nhiên}**

Nhằm xây dựng cơ sở cho quá trình phân tích và thiết kế hệ thống ở các chương sau, phần này mô tả hệ thống EvoDraw dưới dạng ngôn ngữ tự nhiên — bao gồm phạm vi, người dùng, các đối tượng dữ liệu, quan hệ giữa chúng, và mô tả nghiệp vụ chi tiết cho từng chức năng.

#### **Phạm vi phần mềm:**

Phần mềm dạng ứng dụng web, truy cập qua trình duyệt, không yêu cầu đăng ký tài khoản hay đăng nhập. Bất kỳ ai có trình duyệt đều có thể sử dụng. Ngoài ra, có một ứng dụng desktop hỗ trợ tính năng chia sẻ màn hình từ phía người trình bày. Dữ liệu phòng và bảng trắng được lưu trữ tập trung tại server.

#### **Người dùng và chức năng:**

- Hệ thống không phân biệt vai trò đăng nhập, chỉ có một loại tác nhân chính là Người dùng (User) - bất kỳ ai truy cập EvoDraw.
- Người dùng có thể thực hiện các chức năng:
  - Tạo phòng mới (hệ thống tự sinh mã phòng 6 ký tự và mã PIN 4 chữ số)
  - Tham gia phòng hiện có bằng mã phòng và mã PIN hoặc link.
  - Xem danh sách thành viên đang trong phòng
  - Rời khỏi phòng
  - Vẽ tự do lên bảng trắng bằng các công cụ (bút, hình chữ nhật, hình tròn, đường thẳng, mũi tên, văn bản, tẩy)
  - Chia sẻ mã phòng, mã pin và link.
  - Xóa/chỉnh sửa nội dung trên bảng trắng
  - Undo/Redo các thao tác vẽ
  - Phóng to/thu nhỏ bảng trắng
  - Xem con trỏ chuột của thành viên khác theo thời gian thực
  - Chèn ảnh từ thiết bị hoặc clipboard vào bảng trắng
  - Chia sẻ màn hình cho các thành viên (yêu cầu Desktop App)
  - Vẽ/ghi chú trực tiếp lên luồng video đang chia sẻ

- Dừng chia sẻ màn hình
- Gọi thoại (voice chat) qua máy chủ SFU (LiveKit)
- Gửi tin nhắn văn bản (chat) với các thành viên trong phòng
- Lưu bảng trắng thành file (JSON)
- Import trạng thái bảng trắng từ file JSON đã lưu

### Yêu cầu chức năng:

| Nhóm             | Mô tả yêu cầu                                                                                                                                                                 |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quản lý phòng    | Người dùng có thể tạo một phòng mới; hệ thống tự sinh mã phòng 6 ký tự và mã PIN 4 chữ số, trả về cho người dùng để chia sẻ                                                   |
|                  | Người dùng có thể tham gia phòng hiện có bằng mã phòng và mã PIN                                                                                                              |
|                  | Người dùng có thể xem danh sách thành viên đang trong phòng                                                                                                                   |
|                  | Người dùng có thể rời khỏi phòng                                                                                                                                              |
| Bảng trắng       | Người dùng có thể vẽ tự do bằng các công cụ vẽ (bút, hình chữ nhật, hình tròn, đường thẳng, mũi tên, văn bản)                                                                 |
|                  | Người dùng có thể xóa hoặc chỉnh sửa nội dung trên bảng trắng                                                                                                                 |
|                  | Người dùng có thể undo/redo các thao tác vẽ                                                                                                                                   |
|                  | Người dùng có thể phóng to/thu nhỏ bảng trắng                                                                                                                                 |
| Đồng bộ          | Người dùng có thể chèn ảnh từ thiết bị hoặc clipboard vào bảng trắng                                                                                                          |
|                  | Nội dung vẽ được đồng bộ thời gian thực cho tất cả thành viên trong phòng                                                                                                     |
| Chia sẻ màn hình | Vị trí con trỏ chuột của mỗi thành viên hiển thị cho tất cả người khác theo thời gian thực                                                                                    |
|                  | Người dùng có thể chia sẻ màn hình cho các thành viên (thông qua Desktop App)                                                                                                 |
|                  | Thành viên khác có thể vẽ/ghi chú trực tiếp lên luồng màn hình đang chia sẻ                                                                                                   |
|                  | Người dùng có thể dừng chia sẻ màn hình bất kỳ lúc nào                                                                                                                        |
| Gọi thoại        | Người dùng có thể gọi thoại (voice chat) với các thành viên trong phòng                                                                                                       |
| Lưu trữ          | Người dùng có thể lưu bảng trắng thành file (PNG/PDF/JSON)                                                                                                                    |
|                  | Người dùng có thể import trạng thái bảng trắng từ file JSON đã lưu                                                                                                            |
| Chat tin nhắn    | Người dùng có thể gửi tin nhắn văn bản tới tất cả thành viên trong phòng; tin nhắn hiển thị tức thì cho mọi người và được thông báo bằng badge/toast khi khung chat đang đóng |

Bảng 1.2: Yêu cầu chức năng của hệ thống EvoDraw {#bảng-1.2:-yêu-cầu-chức-năng-của-hệ-thống-evodraw}

### Yêu cầu phi chức năng:

| Loại             | Mô tả yêu cầu                                                                                              |
|------------------|------------------------------------------------------------------------------------------------------------|
| Hiệu năng        | Độ trễ đồng bộ nét vẽ và con trỏ chuột giữa các thành viên phải dưới 200ms                                 |
| Khả dụng         | Hệ thống hoạt động trên các trình duyệt phổ biến (Chrome, Firefox, Edge, Safari) mà không cần cài đặt thêm |
| Bảo mật          | Mã PIN phòng phải được mã hóa trước khi lưu trữ, không lưu dạng văn bản thuần                              |
| Khả năng mở rộng | Hệ thống hỗ trợ tối thiểu 10 thành viên đồng thời trong một phòng                                          |
| Tính sẵn sàng    | Phòng không hoạt động trong 24 giờ sẽ tự động bị xóa để giải phóng tài nguyên                              |
| Trải nghiệm      | Người dùng có thể tham gia phòng nhanh chóng mà không cần đăng ký tài khoản hay đăng nhập                  |
| Tương thích      | Tính năng chia sẻ màn hình yêu cầu cài đặt Desktop App (hỗ trợ Windows)                                    |

Bảng 1.3: Yêu cầu phi chức năng của hệ thống EvoDraw {#bảng-1.3:-yêu-cầu-phi-chức-năng-của-hệ-thống-evodraw}

### Thông tin các đối tượng cần xử lý:

- Thông tin một phòng họp: mã phòng (6 ký tự ngẫu nhiên), mã PIN (4 chữ số), trạng thái (active, inactive), thời điểm tạo, thời điểm tương tác gần nhất, mảng phần tử canvas, phiên bản canvas, trạng thái giao diện chia sẻ.
- Thành viên: tên hiển thị, mã socket, phòng đang tham gia.
- Phần tử canvas: loại (path/rect/circle/line/text/image), tọa độ, thuộc tính đồ họa (màu, nét, kích thước), định danh phần tử, phiên bản, khóa ngẫu nhiên giải xung đột.
- File: định danh logic, mã phòng tham chiếu, loại MIME, URL công khai trên bộ nhớ cloud, kích thước, thời điểm tải lên, thời điểm truy cập gần nhất.
- Con trỏ chuột: tọa độ x/y, tên thành viên.
- Tin nhắn: người gửi, nội dung, thời điểm gửi.

### Quan hệ giữa các đối tượng:

- Một phòng họp có thể chứa nhiều thành viên tại một thời điểm, mỗi thành viên chỉ thuộc một phòng họp duy nhất.
- Một phòng họp chứa nhiều phần tử canvas, mỗi phần tử canvas thuộc duy nhất một phòng họp.
- Một phòng họp có thể có nhiều tập tin (file), mỗi tập tin (file) chỉ thuộc một phòng họp duy nhất.
- Một thành viên chỉ có một con trỏ chuột tương ứng duy nhất tại mỗi thời điểm.
- Một thành viên có thể gửi nhiều tin nhắn trong một phòng họp; mỗi tin nhắn gắn với đúng một thành viên tại thời điểm gửi và được gửi cho tất cả thành viên khác trong cùng phòng.
- Phòng tự động bị xóa sau 24 giờ không có tương tác, kéo theo xóa các phần tử canvas nhúng và các tập tin trong phòng cùng bị xóa theo.

### Mô tả nghiệp vụ chi tiết các chức năng:

- **Chức năng tạo phòng mới:** Người dùng truy cập trang chủ → nhấn "Tạo phòng" → hệ thống tự sinh mã phòng 6 ký tự ngẫu nhiên (đảm bảo duy nhất trong CSDL) và mã PIN 4 chữ số ngẫu nhiên, băm PIN bằng bcrypt rồi lưu vào CSDL → chuyển người dùng vào giao diện bảng trắng → hiển thị cặp mã phòng & PIN dưới dạng văn bản để người dùng sao chép và chia sẻ cho đồng nghiệp.
- **Chức năng tham gia phòng:** Người dùng nhập mã phòng và PIN tại trang chủ → nhấn "Tham gia" → hệ thống xác thực mã phòng và PIN → nếu đúng: chuyển vào giao diện phòng, tải nội dung bảng trắng hiện tại → nếu sai: thông báo lỗi, yêu cầu nhập lại.
- **Chức năng rời khỏi phòng:** Người dùng nhấn nút "Rời phòng" trên giao diện phòng → hệ thống hiển thị hộp thoại xác nhận yêu cầu người dùng xác nhận hành động → nếu người dùng xác nhận: hệ thống ngắt kết nối Socket.IO của người dùng khỏi phòng, xóa thông tin thành viên ra khỏi danh sách thành viên của phòng, ngừng phát tín hiệu con trỏ chuột và luồng âm thanh (nếu đang bật micro), đồng thời thông báo cho tất cả thành viên còn lại trong phòng rằng người dùng đã rời đi → cập nhật danh sách thành viên hiển thị trên giao diện của các thành viên còn lại → chuyển người dùng quay về trang chủ. Nếu người dùng là thành viên cuối cùng trong phòng, phòng vẫn được giữ nguyên trạng thái trên server (không bị xóa ngay) cho đến khi hết 24 giờ không có tương tác thì mới tự động xóa. Nếu người dùng đóng tab trình duyệt hoặc mất kết nối mạng mà không nhấn "Rời phòng", hệ thống tự động phát hiện sự kiện ngắt kết nối (disconnect) và thực hiện quy trình tương tự như khi người dùng chủ động rời phòng.
- **Chức năng vẽ trên bảng trắng:** Người dùng chọn công cụ vẽ từ thanh công cụ (bút, hình, văn bản...) → tùy chỉnh màu sắc, độ dày nét → vẽ trên vùng bảng trắng → nội dung hiển thị ngay lập tức → hệ thống tự động đồng bộ nét vẽ cho tất cả thành viên khác trong phòng theo thời gian thực.
- **Chức năng xem con trỏ chuột:** Người dùng di chuyển chuột trên bảng trắng → vị trí con trỏ được hiển thị cho tất cả thành viên khác theo thời gian thực → mỗi con trỏ kèm tên thành viên tương ứng.
- **Chức năng chèn ảnh:** Người dùng chọn "Chèn ảnh" từ thanh công cụ hoặc dán từ clipboard (Ctrl+V) → chọn file ảnh từ thiết bị → hệ thống tải ảnh lên và hiển thị trên bảng trắng → ảnh được đồng bộ cho các thành viên. Nếu file không hỗ trợ hoặc quá lớn → thông báo lỗi.
- **Chức năng chia sẻ màn hình:** Người dùng nhấn "Chia sẻ màn hình" → hệ thống mở ứng dụng Desktop App → hiển thị danh sách cửa sổ/màn hình → người dùng chọn nguồn muốn chia sẻ → luồng video được phát trực tiếp cho các thành viên trong phòng. Người xem có thể vẽ/ghi chú lên luồng video. Người chia sẻ có thể dừng bất kỳ lúc nào.
- **Chức năng gọi thoại:** Người dùng nhấn bật micro → trình duyệt yêu cầu quyền truy cập micro → sau khi cấp quyền, âm thanh được truyền trực tiếp tới các thành viên khác → người dùng có thể bật/tắt micro bất kỳ lúc nào.
- **Chức năng gửi tin nhắn chat:** Người dùng mở khung Chat ở góc phòng → nhập nội dung tin nhắn vào ô input và nhấn Enter (hoặc nút Gửi) → tin nhắn được hiển thị ngay lập tức tại khung chat của người gửi, đồng thời gửi qua Socket.IO tới server → server phát tin nhắn tới tất cả các thành viên khác trong cùng phòng → thành viên nhận được tin nhắn thấy nó xuất hiện trong khung chat, nếu khung chat của họ đang đóng thì một toast thông báo và badge đếm số tin chưa đọc sẽ hiển thị cạnh nút mở chat. Tin nhắn không được lưu trong CSDL, chỉ tồn tại trong phiên hiện tại của từng client.
- **Chức năng export bảng trắng:** Người dùng mở Settings → chọn "Xuất file" → chọn định dạng (JSON) → hệ thống tạo file và kích hoạt tải xuống → file được lưu trực tiếp xuống thiết bị.
- **Chức năng import bảng trắng:** Người dùng chọn file JSON đã lưu trước đó → hệ thống đọc file và khôi phục trạng thái bảng trắng → nội dung được đồng bộ cho tất cả thành viên trong phòng.

## 1.4. Xây dựng sơ đồ usecase {#1.4.-xây-dựng-sơ-đồ-usecase}

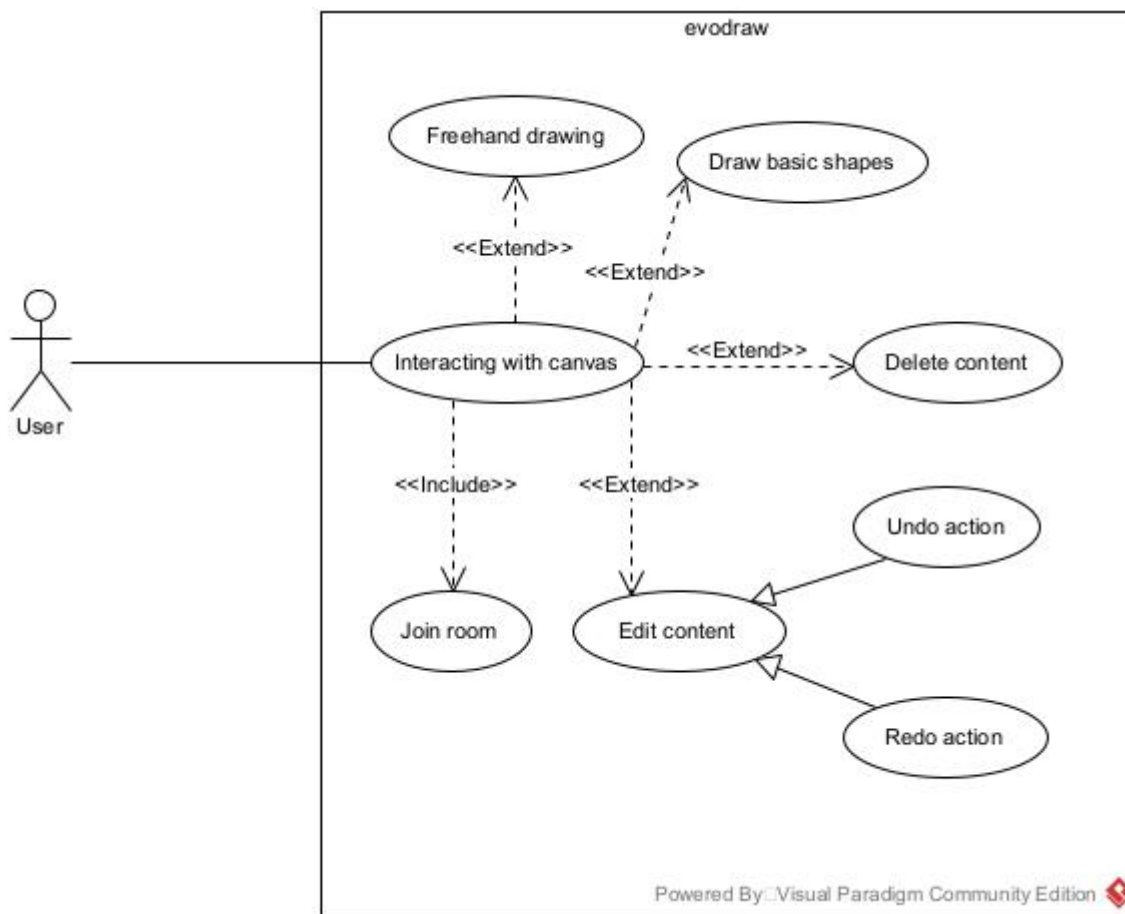
### 1. Sơ đồ usecase tổng quan



Hình 1.1: Sơ

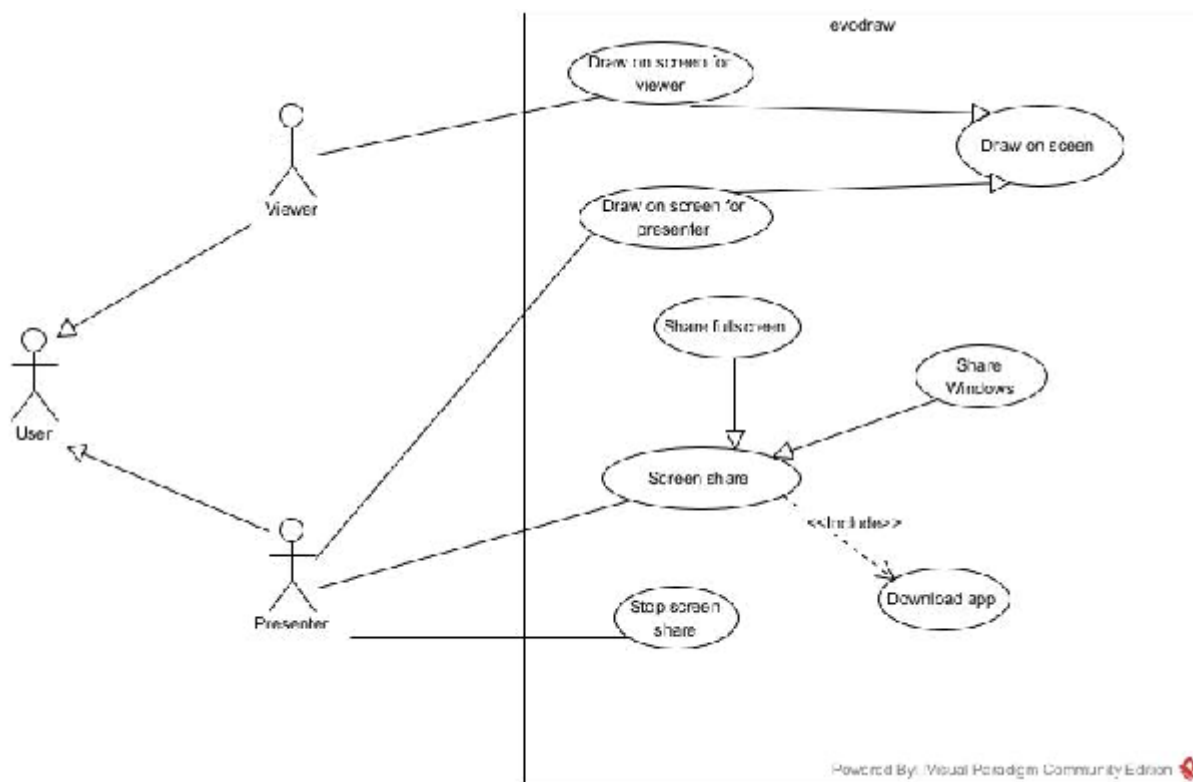
đồ Use Case tổng quan của hệ thống EvoDraw {#hình-1.1:-sơ-đồ-use-case-tổng-quan-của-hệ-thống-evodraw}

### 2. Phân rã chi tiết các usecase



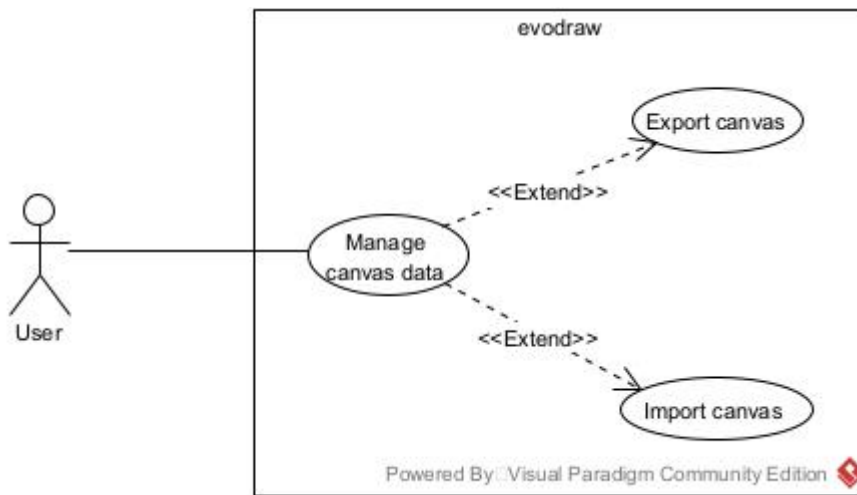
Hình 1.2: Sơ đồ

Use Case phân rã — Vẽ lên bảng trắng {#hình-1.2:-sơ-đồ-use-case-phân-rã—vẽ-lên-bảng-trắng}



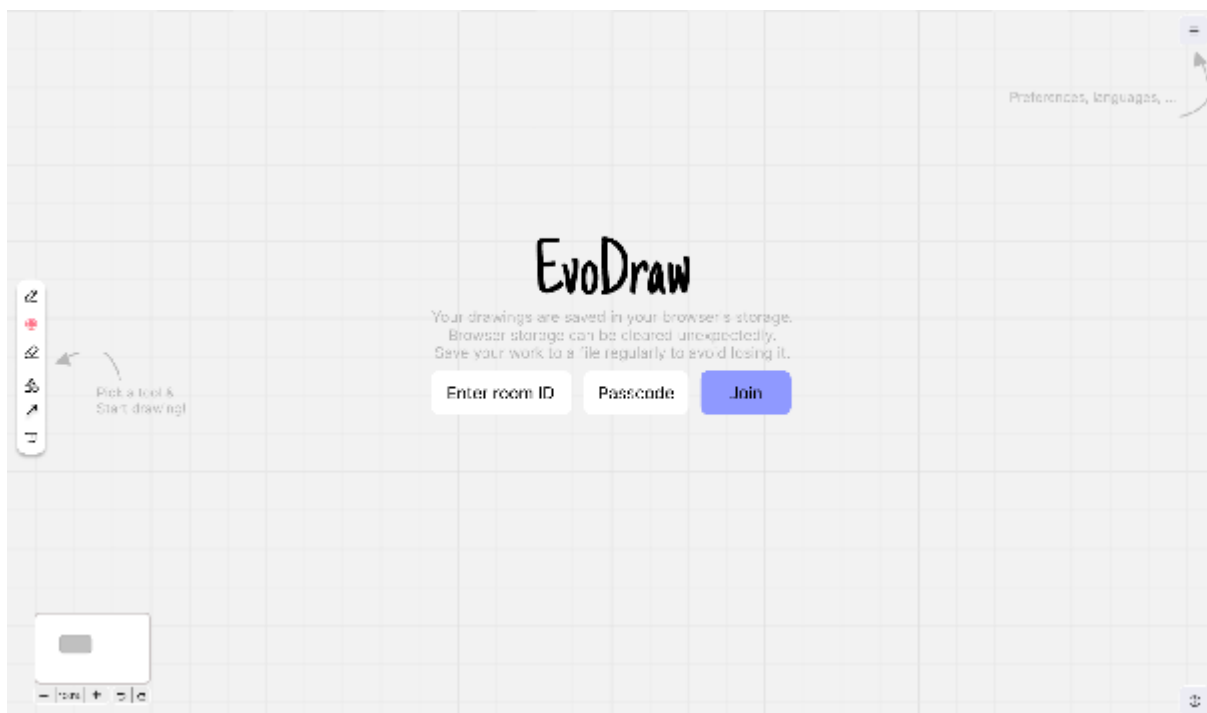
Hình 1.3: Sơ

đồ Use Case phân rã — Chia sẻ màn hình {#hình-1.3:-sơ-đồ-use-case-phân-rã—chia-sẻ-màn-hình}



Hình 1.4: Sơ đồ Use Case phân rã — Quản lý dữ liệu canvas {#hình-1.4:-sơ-đồ-use-case-phân-rã---quản-lý-dữ-liệu-canvas}

## 1.5 Thiết kế tương tác cho sản phẩm {#1.5-thiết-kế-tương-tác-cho-sản-phẩm}



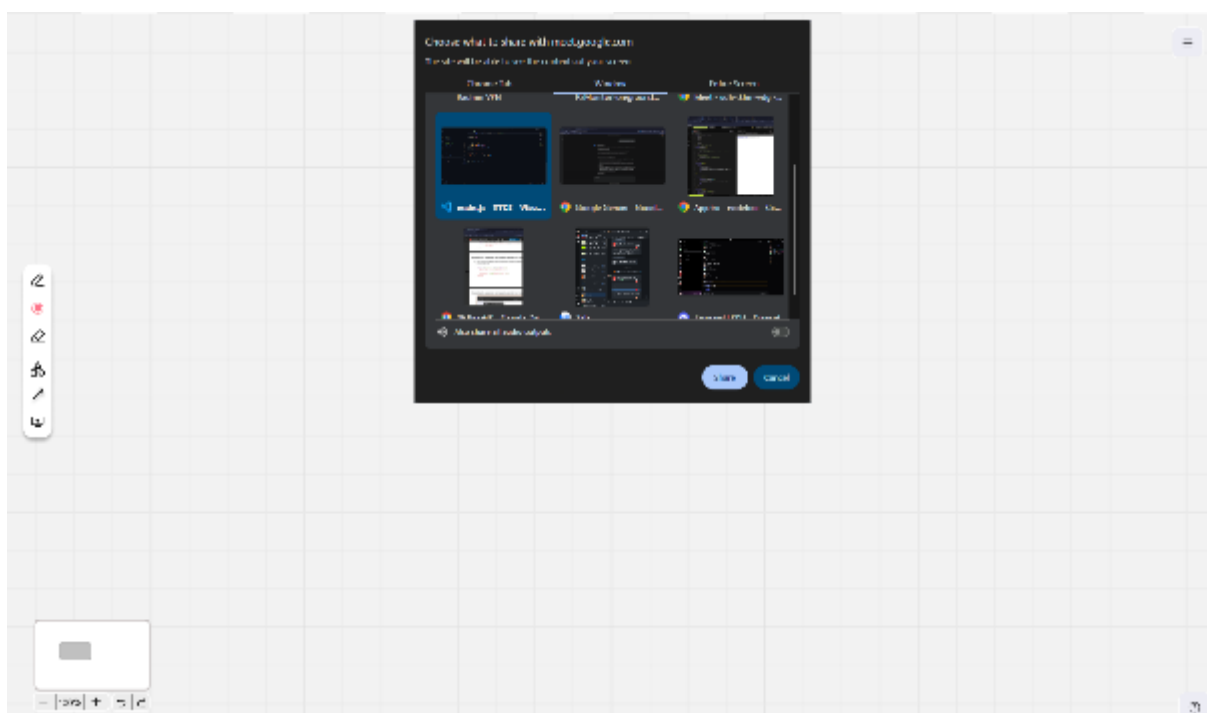
Hình 1.6:

Thiết kế giao diện Trang chủ (Tạo / Tham gia phòng) {#hình-1.6:-thiết-kế-giao-diện-trang-chủ-(tạo-/-tham-gia-phòng)}



Hình 1.7:

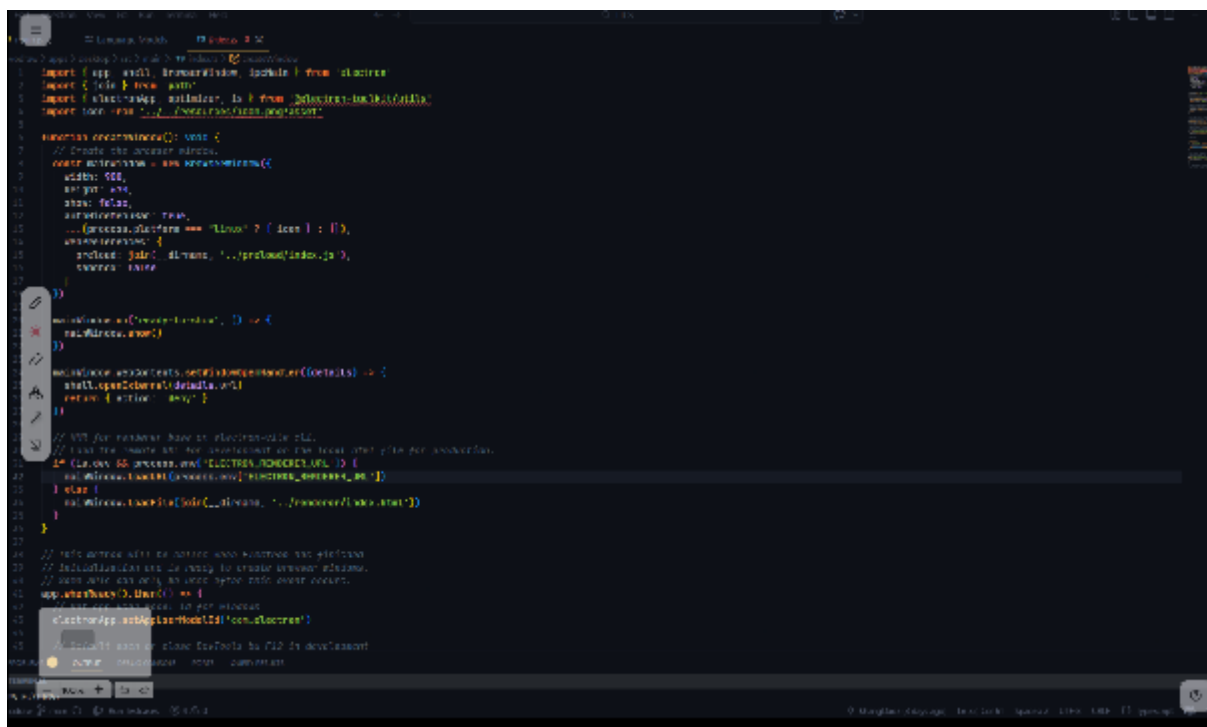
Thiết kế giao diện Phòng làm việc (bảng trắng + thanh công cụ) {#hình-1.7:-thiết-kế-giao-diện-phòng-làm-việc-(bảng-trắng-+-thanh-công-cụ)}



Hình 1.8:

Thiết kế giao diện Chia sẻ màn hình {#hình-1.8:-thiết-kế-giao-diện-chia-sẻ-màn-hình}





Hình 1.9:

Thiết kế giao diện Cài đặt / Tùy chỉnh phòng {#hình-1.9:-thiết-kế-giao-diện-cài-đặt-/-tùy-chỉnh-phòng}

Link to Figma: [EvoDraw](#)

## CHƯƠNG 2 : NGHIÊN CỨU PHƯƠNG PHÁP TIẾP CẬN VÀ GIẢI QUYẾT VẤN ĐỀ {#chương-2:-nghiên-cứu-phương-pháp-tiếp-cận-và-giải-quyết-vấn-đề}

Chương 1 đã xác định rõ bài toán cần giải quyết cùng với yêu cầu hệ thống cụ thể dành cho người dùng, đồng thời phân tích ưu nhược điểm của các giải pháp hiện có trên thị trường. Trên cơ sở đó, Chương 2 tập trung trình bày phương pháp tiếp cận mà nhóm lựa chọn để xây dựng hệ thống EvoDraw - bao gồm mô hình tổng quát, phương pháp luận phát triển phần mềm, kiến trúc hệ thống, và lý do lựa chọn từng công nghệ cụ thể gắn với từng thành phần (module) trong kiến trúc.

### 2.1. Mô hình tổng quát hệ thống {#2.1.-mô-hình-tổng-quát-hệ-thống}

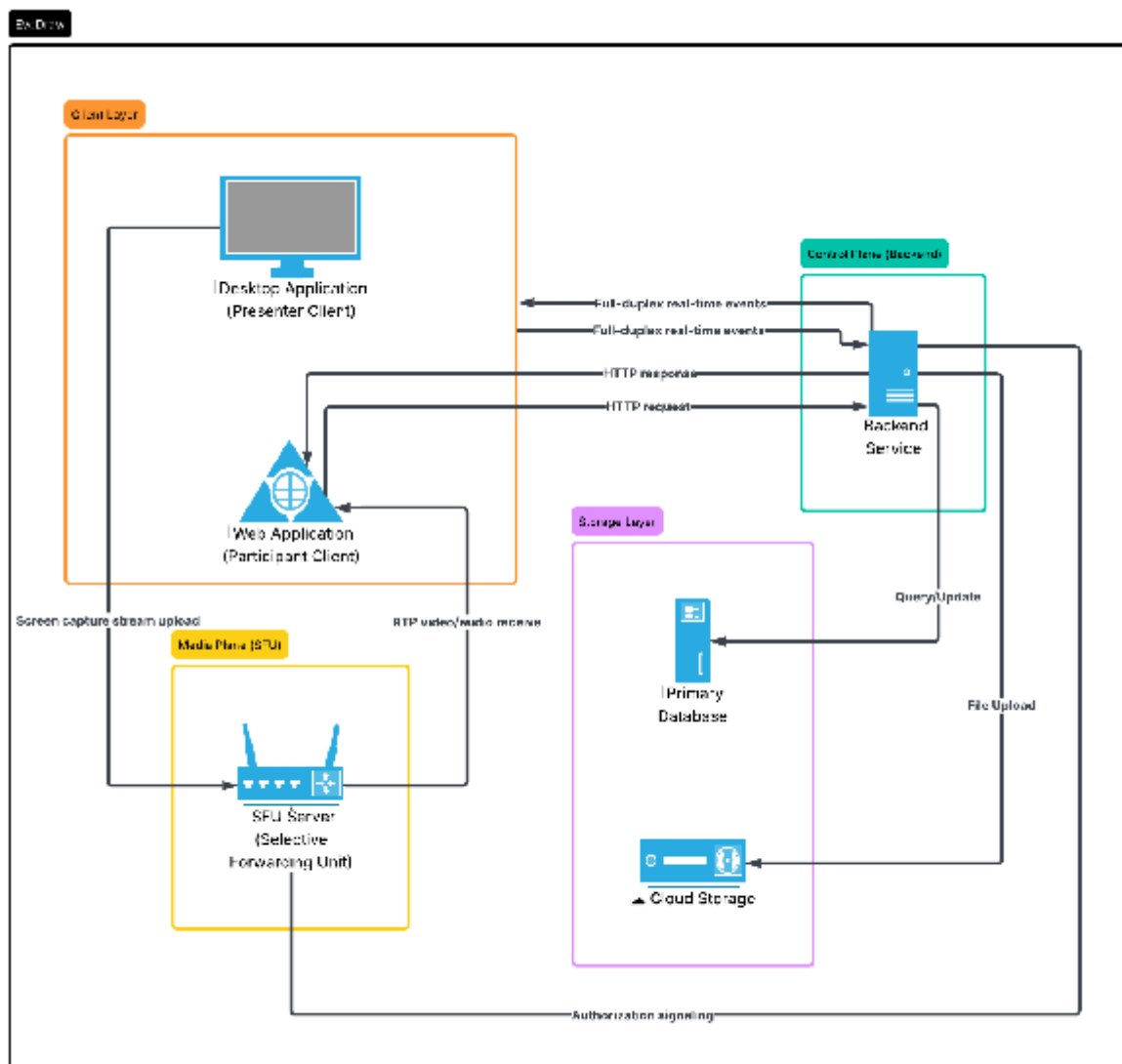
#### 2.1.1. Tổng quan hệ thống {#2.1.1.-tổng-quan-hệ-thống}

EvoDraw là một nền tảng không gian làm việc trực tuyến, cung cấp môi trường cộng tác bảng vẽ thời gian thực và giao tiếp đa phương tiện (video, voice call, chat). Thay vì chỉ quản lý và lưu trữ thông tin cơ bản, hệ thống EvoDraw thực hiện nhiệm vụ cốt lõi là đồng bộ hóa trạng thái, điều phối liên tục các thao tác trực quan trên bảng trắng và định tuyến luồng dữ liệu thời gian thực cho các phòng họp.

Hệ thống bao gồm các thành phần phối hợp chặt chẽ với nhau:

- **Web App:** Đóng vai trò là Client chính dành cho người tham gia. Phần lớn logic xử lý đồ họa vector và đối chiếu trạng thái được đặt tại Client nhằm đảm bảo phản hồi tức thì cho người dùng, tự động xử lý giải quyết xung đột khi có nhiều người cùng thao tác.
- **Desktop App:** Đóng vai trò là ứng dụng hỗ trợ người trình bày, cung cấp khả năng can thiệp sâu vào tài nguyên hệ thống máy tính. Module này cho phép quay màn hình với độ trễ thấp và tạo lớp phủ (overlay) trong suốt trên cùng, giúp người dùng có thể vẽ và ghi chú trực tiếp đè lên bất kỳ ứng dụng nào khác đang mở trên toàn hệ thống.
- **Backend Service:** Đóng vai trò là "người điều phối" trung tâm, máy chủ không trực tiếp can thiệp nội dung bản vẽ mà chỉ chịu trách nhiệm xác thực, lưu vết dữ liệu vào cơ sở dữ liệu và phát sóng các sự kiện thời gian thực ( nét vẽ, tọa độ con trỏ chuột, tin nhắn) đến toàn bộ người dùng kết nối với độ trễ thấp.
- **SFU Server:** Để giải quyết bài toán nghẽn băng thông của mạng ngang hàng truyền thống, luồng dữ liệu nặng (âm thanh và video chia sẻ màn hình) được tách biệt hoàn toàn và giao cho kiến trúc máy chủ SFU định tuyến. Điều này giúp giảm tải tuyệt đối cho Backend chính và hỗ trợ phòng họp mở rộng linh hoạt.

### 2.1.2. Sơ đồ kiến trúc tổng quát {#2.1.2.-sơ-đồ-kiến-trúc-tổng-quát}



Hình 2.1: Sơ

đồ tổng quát hệ thống EvoDraw {#hình-2.1:-sơ-đồ-tổng-quát-hệ-thống-evodraw}

### 2.1.3. Các luồng luân chuyển dữ liệu chính {#2.1.3.-các-luồng-luân-chuyển-dữ-liệu-chính}

Để đảm bảo hiệu năng và tính ổn định, hệ thống không gộp chung dữ liệu mà phân tách kiến trúc luân chuyển thành 4 luồng độc lập dựa trên đặc thù về tần suất và dung lượng:

- **Luồng dữ liệu trạng thái và cấu hình (HTTP Request-Response):** Đây là luồng giao tiếp chuẩn giữa Client và API Server để xử lý các yêu cầu không mang tính liên tục. Luồng này chịu trách nhiệm xác thực mã phòng, cấp phát phiên làm việc, và truy xuất dữ liệu của bảng trắng từ Cơ sở dữ liệu khi người dùng mới tham gia.
- **Luồng sự kiện thời gian thực (Full-duplex):** Hoạt động liên tục với tần suất cực cao nhưng dung lượng payload rất nhỏ. Luồng này vận chuyển tọa độ chuột, thao tác thêm/xóa/sửa nét vẽ và tin nhắn chat. Máy chủ đóng vai trò trung tâm phân phối, định tuyến gói tin ngay trong bộ nhớ đệm đến các thành viên cùng phòng mà không ghi đè liên tục xuống Database.
- **Luồng định tuyến đa phương tiện (Real-time Media Streaming):** Các dữ liệu nặng về băng thông luồng video như chia sẻ màn hình và âm thanh Voice Chat được tách riêng biệt. Dữ liệu truyền từ thiết bị lên một máy chủ trung tâm chuyên trách điều phối media (Media Server). Máy chủ này nhận luồng gốc và phân phối độc lập tới từng người xem, khắc phục triệt để nhược điểm sập mạng của mô hình mạng ngang hàng (P2P) truyền thống khi phòng có đông người.
- **Luồng tệp tin tĩnh (Object Storage):** Đối với chức năng chèn ảnh tĩnh, Client không truyền trực tiếp dữ liệu nhị phân qua luồng thời gian thực để tránh làm phình to payload. File ảnh được tải qua API Server để kiểm duyệt định dạng, sau đó đẩy lưu trữ tại hệ thống đám mây. Trình duyệt sau đó chỉ sử dụng URL công khai trả về để đồng bộ hiển thị.

### 2.1.4. Cơ chế đồng bộ và chiến lược giải quyết xung đột {#2.1.4.-cơ-chế-đồng-bộ-và-chiến-lược-giải-quyết-xung-đột}

Việc nhiều thành viên cùng tương tác vào một không gian chung đòi hỏi chiến lược xử lý đồng thời khắt khe. Hệ thống giải quyết sự cố ghi đè và đồng bộ hóa thông qua 4 cơ chế cốt lõi:

- **Cơ chế máy chủ chuyển tiếp (Relay Server) và ủy quyền xử lý cho Client:** Máy chủ backend không tham gia phân tích, tính toán hay render cấu trúc hình học của bảng trắng. Mọi thao tác vẽ được Engine đồ họa ở Client đóng gói thành đối tượng dữ liệu. Máy chủ chỉ làm nhiệm vụ lưu trữ nguyên bản và chuyển tiếp mảng cấu trúc này đến các kết nối khác. Thiết kế này giải phóng CPU của máy chủ, cho phép hệ thống dễ dàng mở rộng để chịu tải quy mô lớn.
- **Giải quyết xung đột qua chiến lược Last-Writer-Wins (LWW):** Mỗi phần tử trên bảng trắng được định danh bởi một ID duy nhất và đi kèm một bộ đếm phiên bản (version). Khi hai Client cùng thao tác lên một đối tượng, phía nhận sẽ so sánh phiên bản của đối tượng. Hành động có phiên bản lớn hơn sẽ luôn thắng. Nếu phiên bản bằng nhau, hệ thống sử dụng định danh Client ID làm biến số chốt để đảm bảo tính nhất quán tuyệt đối.
- **Tối ưu băng thông bằng chiến lược Debounce và Throttling:** Tọa độ con trỏ chuột không được gửi ngay lập tức mỗi khi di chuyển mà bị kiểm soát tần suất ở một ngưỡng tối đa (VD: 20-30 lần/giây) để tránh network flooding. Các tác vụ phức tạp như kéo giãn, đổi màu nét vẽ chỉ kích hoạt broadcast khi người dùng kết thúc thao tác hoặc qua một khoảng trễ an toàn.
- **Đối chiếu và hợp nhất trạng thái (State Reconciliation):** Khi một Client bị rớt mạng tạm thời và khôi phục kết nối, nó sẽ tự động lấy snapshot mới nhất từ máy chủ để tiến hành so khớp với các

ID phần tử hiện có cục bộ. Những thao tác cục bộ chưa kịp gửi đi sẽ được tái đồng bộ vào hệ thống dựa trên version, giúp đạt được trạng thái nhất quán cuối cùng mà không làm mất dữ liệu của người dùng.

## 2.2 Phương pháp xây dựng phần mềm {#2.2-phương-pháp-xây-dựng-phần-mềm}

Đề tài áp dụng phương pháp phân tích và thiết kế hướng đối tượng OOAD xác định các thực thể tồn tại rõ ràng như Room, Canvas,... Để hiện thực hóa phương pháp OOAD, đề tài sử dụng ngôn ngữ mô hình hóa thống nhất (UML) với các biểu đồ:

- **Use Case Diagram:** Để xác định và làm rõ các yêu cầu chức năng của hệ thống từ góc nhìn của người dùng.
- **Sequence Diagram:** Để mô tả chi tiết sự tương tác theo trình tự thời gian giữa các đối tượng.
- **Class Diagram:** Để thể hiện cấu trúc tĩnh của hệ thống, bao gồm các lớp đối tượng, thuộc tính, phương thức, và quan hệ giữa chúng.

Sử dụng phương pháp trên giúp cấu trúc bài toán trở nên rõ ràng, dễ thiết kế, dễ giao tiếp giữa các thành viên trong nhóm, và thuận lợi cho việc lập trình cũng như bảo trì sau này. Các biểu đồ UML cụ thể sẽ được trình bày chi tiết trong Chương 3 (Phân tích thiết kế hệ thống).

## 2.3 Mô hình phát triển phần mềm {#2.3-mô-hình-phát-triển-phần-mềm}

Đề tài áp dụng mô hình phát triển phần mềm Scrum dựa trên nền tảng Lặp và Tăng trưởng (Iterative and Incremental). Việc lựa chọn mô hình này giúp dự án dễ dàng thích ứng với các thay đổi yêu cầu và đảm bảo liên tục tạo ra các phiên bản phần mềm hoạt động được sau mỗi giai đoạn ngắn.

Cách thức vận hành:

- Toàn bộ quá trình phát triển được chia thành các vòng lặp ngắn gọi là Sprint, mỗi Sprint được ấn định thời gian cố định là 2 tuần.
- Trong mỗi Sprint, nhóm thực hiện đầy đủ các luồng công việc: Phân tích, Thiết kế, Thực thi và Kiểm thử cho tính năng mục tiêu.
- Nhóm áp dụng kỹ thuật họp đứng hàng ngày để đồng bộ tiến độ, giao tiếp cường độ cao và tháo gỡ khó khăn kịp thời.

Quá trình xây dựng sản phẩm được chia thành các phần tăng trưởng cụ thể như sau:

- **Sprint 1 (Tuần 1-2) - "Foundation" (Nền tảng):** Tập trung vào việc phân tích yêu cầu, nghiên cứu cơ sở lý thuyết về các kiến trúc hệ thống phân tán, các giao thức truyền tải thời gian thực và công nghệ xử lý đồ họa vector trên trình duyệt. Tiến hành thiết kế hệ thống bao gồm kiến trúc tổng thể, cơ sở dữ liệu và phác thảo giao diện.
  - => **Kết quả (Milestone 1):** Hiểu rõ cơ chế hoạt động của các luồng dữ liệu thời gian thực, hoàn thiện các tài liệu thiết kế cơ sở và thiết lập sẵn sàng môi trường lưu trữ mã nguồn.
- **Sprint 2 (Tuần 3-4) - "Core Services & Room System" (Dịch vụ lõi):** Khởi tạo khung dự án cho các ứng dụng đầu cuối (Client) và bộ phận xử lý trung tâm (Server). Triển khai hệ thống quản lý không gian làm việc (Room model), xây dựng các API giao tiếp chuẩn và kênh kết nối song công với cơ chế bảo mật, định danh tự động.

- => **Kết quả (Milestone 2):** Hoàn thiện chức năng tạo phòng (cấp tự động mã truy cập và mã PIN), tham gia phòng xác thực an toàn, tự động thu dọn tài nguyên đối với phòng không hoạt động sau 24h và đồng bộ danh sách thành viên trực tuyến.
- **Sprint 3 (Tuần 5–6) - "Whiteboard Real-time" (Bảng trắng thời gian thực):** Tích hợp và phát triển engine xử lý đồ họa để xây dựng các công cụ tương tác trên bảng trắng. Triển khai cơ chế đồng bộ hóa trạng thái (chuột, nét vẽ, đối tượng) giữa các người dùng thông qua kênh kết nối liên tục, đồng thời xử lý các bài toán về xung đột dữ liệu.
  - => **Kết quả (Milestone 3):** Bảng trắng được đồng bộ mượt mà theo thời gian thực giữa nhiều người dùng, kết hợp với tính năng trò chuyện văn bản (chat) hoạt động ổn định.
- **Sprint 4 (Tuần 7–8) - "Media Routing & Annotation" (Đa phương tiện & Ghi chú):** Triển khai hạ tầng định tuyến đa phương tiện để xử lý các luồng dữ liệu nặng. Xây dựng tính năng chia sẻ màn hình độ trễ thấp và phát triển lớp phủ đồ họa (overlay) cho phép người dùng vẽ ghi chú trực tiếp lên luồng video đang được chia sẻ.
  - => **Kết quả (Milestone 4 - MVP Complete):** Hoàn thiện chức năng chia sẻ màn hình và vẽ đè, đánh dấu mốc toàn bộ các tính năng cốt lõi của Sản phẩm khả thi tối thiểu (MVP) đã hoạt động đầy đủ.
- **Sprint 5 (Tuần 9–10) - "Testing & Expansion" (Kiểm thử & Mở rộng):** Thực hiện kiểm thử toàn diện các tính năng MVP trên đa nền tảng trình duyệt, đo lường hiệu năng chịu tải và khảo sát trải nghiệm người dùng. Đồng thời, tinh chỉnh giao diện và bổ sung các tính năng nâng cao (nếu kịp tiến độ).
  - => **Kết quả (Milestone 5):** Bàn giao sản phẩm phiên bản Ổn định (Release Candidate), đã qua kiểm thử, khắc phục các lỗi phát sinh (bugs) và ghi nhận phản hồi.
- **Sprint 6 (Tuần 11–12) - "Documentation & Defense" (Hoàn thiện & Bảo vệ):** Tổng hợp số liệu, chỉnh sửa báo cáo toàn văn, bổ sung tài liệu phụ lục hướng dẫn sử dụng, chuẩn bị slide thuyết trình và đóng gói bản Demo thực tế triển khai trên máy chủ đám mây.
  - => **Kết quả (Milestone 6):** Sẵn sàng toàn bộ tài liệu báo cáo, bản trình bày và sản phẩm thực tế để bảo vệ đề tài trước hội đồng.

## 2.4 Kiến trúc phần mềm được áp dụng trong triển khai lập trình hệ thống {#2.4-kiến-trúc-phần-mềm-được-áp-dụng-trong-triển-khai-lập-trình-hệ-thống}

Để đảm bảo tính mở rộng, dễ bảo trì và làm nền tảng vững chắc cho việc lựa chọn công nghệ ở giai đoạn sau, hệ thống EvoDraw được thiết kế tuân thủ chặt chẽ các mẫu kiến trúc và nguyên lý sau:

- **Mô hình Client-Server:** Đây là kiến trúc nền tảng chủ đạo, phân tách rõ ràng ranh giới giữa luồng xử lý giao diện người dùng (Client) và luồng xử lý nghiệp vụ, quản lý dữ liệu trung tâm (Server). Việc phân tách này cho phép hai phân hệ tiến hóa độc lập và tối ưu hóa tài nguyên phần cứng tương ứng.
- **Cấu trúc kho lưu trữ tập trung (Monorepo Architecture):** Thay vì phân tán mã nguồn thành nhiều kho lưu trữ riêng lẻ, toàn bộ các phân hệ của dự án (Web App, Desktop App, Backend Service) được quản lý tập trung trong một cấu trúc cây thư mục duy nhất. Cách tiếp cận này tạo điều kiện thuận lợi để cấu hình đồng bộ các tiêu chuẩn kiểm tra mã tĩnh, định dạng mã nguồn, và tái sử dụng các thư viện logic dùng chung giữa các môi trường mà không gây phân mảnh.
- **Kiến trúc phân lớp tại Backend:** Mã nguồn máy chủ được thiết kế cô lập thành các lớp có trách nhiệm đơn lẻ nhằm giảm thiểu sự phụ thuộc chéo:

- **Lớp Định tuyến (Routing Layer):** Chịu trách nhiệm tiếp nhận yêu cầu, định tuyến điểm mù và giới hạn lưu lượng truy cập.
- **Lớp Điều khiển (Controller Layer):** Xử lý luồng dữ liệu giao thức, xác thực định dạng và tính hợp lệ của gói tin.
- **Lớp Nghiệp vụ (Service Layer):** Nơi tập trung toàn bộ logic nghiệp vụ cốt lõi (thuật toán tạo phòng, xác thực, cấp quyền, giải quyết xung đột).
- **Lớp Truy cập Dữ liệu (Data Access Layer):** Trừu tượng hóa việc giao tiếp với cơ sở dữ liệu vật lý thông qua các mô hình lược đồ (Schema).
- **Kiến trúc hướng thành phần tại Frontend:** Ứng dụng Client được module hóa thành các khối giao diện độc lập, khép kín và có khả năng tái sử dụng cao (ví dụ: Bảng vẽ, Thanh công cụ, Khung chat).
  - **Đặc biệt áp dụng Mẫu thiết kế phân tách Trạng thái/Giao diện :** Toàn bộ logic xử lý trạng thái phức tạp (như điều phối kết nối thời gian thực, quản lý luồng đa phương tiện) được bóc tách hoàn toàn khỏi lớp hiển thị (View). Kiến trúc này giúp luồng dữ liệu ở Frontend chảy theo một chiều nhất quán, mã nguồn tinh gọn và đáp ứng tốt việc kiểm thử tự động.

## 2.5. Lựa chọn công nghệ phù hợp để triển khai xây dựng hệ thống {#2.5.-lựa-chọn-công-nghệ-phù-hợp-để-triển-khai-xây-dựng-hệ-thống}

Để hiện thực hóa các chức năng của từng module trong mô hình tổng quát (mục 2.1) và tuân thủ chặt chẽ kiến trúc phần mềm đã đề ra (mục 2.4), các công nghệ sau đây được lựa chọn dựa trên khả năng tối ưu hóa hiệu suất và tính tương thích của hệ thống:

### 2.5.1. Công nghệ cho Module Desktop App {#2.5.1.-công-nghệ-cho-module-desktop-app}

Do đặc thù của module này cần can thiệp sâu vào tài nguyên hệ thống (điều mà trình duyệt web thông thường bị hạn chế), các công nghệ sau đã được lựa chọn:

- **Electron.js:** Được chọn làm nền tảng chính nhờ khả năng kết hợp giữa giao diện Web và quyền hạn của Node.js. Điều này cho phép ứng dụng chạy đa nền tảng nhưng vẫn có thể sử dụng các Native API để tạo cửa sổ không viền, trong suốt.
- **getDisplayMedia API (Web App):** Tính năng quay màn hình thực tế được thực hiện bởi **Web App** (không phải Desktop App) thông qua `navigator.mediaDevices.getDisplayMedia()` của trình duyệt. Desktop App không quay màn hình — nó tạo lớp phủ trong suốt để vẽ ghi chú đè lên màn hình.
- **Transparent Windows & Ignore Mouse Events:** Đây là các thuộc tính cốt lõi của Electron giúp tạo lớp phủ (overlay) luôn hiển thị trên cùng. Việc cấu hình `ignore-mouse-events` cho phép người dùng nhìn thấy nét vẽ nhưng vẫn có thể tương tác (click, scroll) trực tiếp với các phần mềm bên dưới lớp vẽ.

### 2.5.2. Công nghệ cho Module Web App {#2.5.2.-công-nghệ-cho-module-web-app}

Module này đóng vai trò là nơi tiếp nhận thao tác người dùng và hiển thị giao diện đồ họa, yêu cầu tính phản hồi cực nhanh:

- **React.js:** Hỗ trợ xây dựng giao diện theo "Kiến trúc hướng thành phần" đã đề ra. Cơ chế Virtual DOM của React giúp tối ưu hóa việc cập nhật UI khi có hàng loạt sự kiện vẽ gửi đến. Việc sử dụng Custom Hooks hỗ trợ tách biệt logic trạng thái phòng họp ra khỏi mã HTML.
- **Fabric.js:** Được chọn làm Drawing Engine chính nhờ mô hình đối tượng (Object-based Canvas). Khác với thẻ <canvas> thuần, Fabric.js quản lý từng nét vẽ, hình khối như một đối tượng độc lập, cực kỳ tối ưu cho việc Serialize/Deserialize dữ liệu sang dạng JSON để gửi qua máy chủ.
- **LiveKit Client (WebRTC qua SFU):** Để thay thế cho mạng ngang hàng (P2P) truyền thống vốn gây nghẽn cổ chai băng thông, hệ thống sử dụng LiveKit SDK để kết nối với luồng định tuyến đa phương tiện (SFU). Nó chịu trách nhiệm phát và nhận luồng video màn hình/âm thanh với cơ chế mã hóa thích ứng.
- **Socket.IO Client:** Xử lý luồng sự kiện thời gian thực (tọa độ chuột, thao tác vẽ, tin nhắn) thông qua giao thức WebSocket hai chiều.

### 2.5.3. Công nghệ cho Module Backend Service {#2.5.3.-công-nghệ-cho-module-backend-service}

Đóng vai trò là trung tâm điều phối dữ liệu tĩnh và phát sóng sự kiện thời gian thực:

- **Node.js & Express.js:** Đóng vai trò là nền tảng xử lý cho cấu trúc phân lớp. Cơ chế Non-blocking I/O của [Node.js](#) phù hợp với bài toán duy trì hàng ngàn kết nối đồng thời từ các phòng bảng trắng. Express.js cung cấp bộ định tuyến để xây dựng các luồng API RESTful (tạo phòng, tham gia phòng).
- **Socket.IO Server:** Máy chủ WebSocket trung tâm, vận hành song song với Express. Nó quản lý khái niệm "Rooms" trong bộ nhớ ảo, nhận các thao tác vẽ/chuột từ một Client và ngay lập tức phát sóng đến các Client khác trong cùng phòng.
- **MongoDB & Mongoose (ODM):** Sử dụng hệ quản trị CSDL NoSQL (MongoDB) để lưu trữ siêu dữ liệu phòng và cấu trúc mảng JSON phức tạp của Fabric.js. Mongoose được dùng ở lớp Data (Models) để định nghĩa Schema chặt chẽ và tận dụng tính năng TTL (Time-To-Live) tự động dọn dẹp các phòng không hoạt động.
- **Firebase Admin & Multer:** Chịu trách nhiệm cho luồng tệp tin nhị phân. Multer đóng vai trò middleware kiểm duyệt file tải lên trên bộ nhớ tạm, sau đó Firebase Admin đẩy thẳng dữ liệu lên Google Cloud Storage để lấy URL công khai, giảm tải lưu trữ cho máy chủ [Node.js](#).

| Module                | Công nghệ sử dụng          | Vai trò và Chức năng chính                                                                                                  |
|-----------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>1. Desktop App</b> | <b>Electron.js</b>         | Nền tảng chính, kết hợp giao diện Web với quyền hạn lõi hệ thống (Node.js).                                                 |
|                       | <b>getDisplayMedia API</b> | Screen capture thực hiện ở Web App qua getDisplayMedia(), publish lên LiveKit SFU. Desktop App chỉ cung cấp lớp vẽ overlay. |
|                       | <b>Transparent Windows</b> | Tạo không gian vẽ dạng lớp phủ (overlay) trong suốt, luôn hiển thị trên cùng.                                               |
| <b>2. Web App</b>     | <b>React.js</b>            | Xây dựng giao diện hướng thành phần, quản lý vòng đời và trạng thái client.                                                 |

| Module                    | Công nghệ sử dụng                  | Vai trò và Chức năng chính                                                         |
|---------------------------|------------------------------------|------------------------------------------------------------------------------------|
|                           | <b>Fabric.js</b>                   | Engine xử lý đồ họa vector, quản lý các nét vẽ dưới dạng đối tượng thao tác được.  |
|                           | <b>LiveKit Client</b>              | Kết nối máy chủ SFU để truyền tải âm thanh và luồng màn hình ổn định.              |
|                           | <b>Socket.IO Client</b>            | Thiết lập kênh song công nhận/gửi các lệnh vẽ, tọa độ, và tin nhắn thời gian thực. |
| <b>3. Backend Service</b> | <b>Node.js &amp; Express.js</b>    | Cung cấp REST API, áp dụng kiến trúc phân lớp xử lý nghiệp vụ bảo mật phòng.       |
|                           | <b>Socket.IO Server</b>            | Điều phối và phát sóng (broadcast) hàng loạt luồng sự kiện tốc độ cao.             |
|                           | <b>MongoDB &amp; Mongoose</b>      | Lưu trữ NoSQL mảng dữ liệu JSON của bảng trắng, quản lý vòng đời phòng (TTL).      |
|                           | <b>Firebase Admin &amp; Multer</b> | Kiểm duyệt luồng file và lưu trữ đám mây ngoài (Cloud Object Storage).             |

Bảng 2.1. Tổng hợp công nghệ sử dụng cho từng module {#bảng-2.1.-tổng-hợp-công-nghệ-sử-dụng-cho-từng-module}

## 2.6. Kết luận Chương 2 {#2.6.-kết-luận-chương-2}

Chương 2 đã trình bày chi tiết phương pháp luận và cơ sở kỹ thuật cốt lõi để xây dựng hệ thống EvoDraw. Qua việc phân tích, nhóm đã xác lập được:

- Mô hình hệ thống: Lựa chọn kiến trúc Client-Server kết hợp kiến trúc SFU (LiveKit) cho luồng media (âm thanh và video), nhằm tối ưu hóa tốc độ truyền tải và đồng bộ dữ liệu thời gian thực.
- Phương pháp phát triển: Áp dụng quy trình Scrum với 6 Sprint cụ thể, giúp quản lý tiến độ linh hoạt và đảm bảo hoàn thiện sản phẩm theo từng giai đoạn tăng trưởng (Incremental).
- Giải pháp công nghệ: Thiết lập một hệ sinh thái đồng nhất dựa trên JavaScript (Node.js, React, Electron). Việc kết hợp các thư viện chuyên biệt như Fabric.js để xử lý đồ họa và Socket.IO để điều phối sự kiện là lời giải tối ưu cho bài toán tương tác bảng trắng đa nền tảng.

Những nghiên cứu và lựa chọn trong chương này đóng vai trò là kim chỉ nam và nền tảng vững chắc để nhóm tiến hành các bước phân tích, thiết kế chi tiết và hiện thực hóa sản phẩm trong các chương tiếp theo.

## CHƯƠNG 3 : Phân tích, thiết kế và thực nghiệm hệ thống {#chương-3:-phân-tích,-thiết-kế-và-thực-nghiệm-hệ-thống}



## 3.1 Phân tích thiết kế hệ thống: {#3.1-phân-tích-thiết-kế-hệ-thống:}

### 3.1.1. Các kịch bản (Scenarios) cho các Use Case {#3.1.1.-các-kịch-bản-(scenarios)-cho-các-use-case}

#### a. Kịch bản tạo phòng mới {#a.-kịch-bản-tạo-phòng-mới}

1. NSD truy cập trang chủ EvoDraw để bắt đầu phiên làm việc cộng tác mới.
2. Hệ thống hiển thị giao diện trang chủ với các công cụ vẽ: bút, tẩy,...
3. NSD click vào một công cụ bất kỳ.
4. Hệ thống tạo phòng mới, chuyển NSD vào giao diện phòng với bảng trắng trống và hiển thị cặp mã phòng (6 ký tự) và PIN (4 chữ số) để NSD sao chép và chia sẻ.

#### Ngoại lệ:

- Bước 4: Hệ thống không tạo được phòng → hệ thống thông báo lỗi và yêu cầu thử lại.

#### b. Kịch bản tham gia phòng {#b.-kịch-bản-tham-gia-phòng}

1. NSD nhận được cặp mã phòng và PIN (hoặc một liên kết mời).
2. Hệ thống hiển thị tại trang chủ hai ô nhập: Mã phòng (6 ký tự) và Mã PIN (4 chữ số), kèm nút Tham gia.
3. NSD nhập mã phòng và PIN, rồi click nút Tham gia.
4. Hệ thống xác thực thông tin, tải nội dung bảng trắng và chuyển NSD vào giao diện phòng.
5. NSD bắt đầu làm việc trong phòng.
6. Hệ thống hiển thị NSD trong danh sách thành viên cho tất cả người đang có mặt trong phòng.

#### Ngoại lệ:

- Bước 3: Sai định dạng (mã phòng không đúng 6 ký tự hoặc PIN không phải 4 chữ số) → hệ thống từ chối và báo lỗi định dạng.
- Bước 4: Mã phòng không tồn tại (bao gồm phòng đã hết hạn sau 24 giờ không hoạt động) hoặc PIN sai → hệ thống thông báo "Mã phòng hoặc PIN không hợp lệ".

#### c. Kịch bản vẽ tự do trên bảng trắng {#c.-kịch-bản-vẽ-tự-do-trên-bảng-trắng}

1. NSD đang ở trong phòng, muốn vẽ minh họa ý tưởng lên bảng trắng.
2. Hệ thống hiển thị thanh công cụ với các công cụ: bút, tẩy, hình chữ nhật, hình tròn, đường thẳng, mũi tên, văn bản.
3. NSD chọn công cụ bút, chọn màu, độ dày nét, độ trong suốt và kiểu nét (liền/đứt).
4. Hệ thống cập nhật trạng thái công cụ theo lựa chọn của NSD.
5. NSD nhấn giữ chuột và kéo trên vùng bảng trắng.
6. Hệ thống ghi lại và hiển thị nét vẽ theo thời gian thực.
7. NSD thả chuột để kết thúc nét vẽ.
8. Hệ thống hoàn tất và đồng bộ nét vẽ tới tất cả thành viên khác trong phòng.
9. Các thành viên khác thấy nét vẽ hiện lên trên bảng trắng của họ gần như tức thời.

#### Ngoại lệ:

- Bước 8: Mất kết nối mạng → nét vẽ vẫn hiển thị cục bộ cho NSD nhưng không đồng bộ ra ngoài; sau khi kết nối lại, hệ thống tự động hợp nhất các thay đổi.

#### **d. Kịch bản chèn ảnh vào bảng trắng {#d.-kịch-bản-chèn-ảnh-vào-bảng-trắng}**

1. NSD đang ở trong phòng, muốn chèn một hình ảnh minh họa lên bảng trắng.
2. Hệ thống hiển thị thanh công cụ với tùy chọn chèn ảnh.
3. NSD thực hiện một trong hai cách: (a) dán ảnh từ bộ nhớ đệm bằng Ctrl+V, hoặc (b) chọn file ảnh từ thiết bị qua thanh công cụ.
4. Hệ thống kiểm tra file ảnh (chỉ chấp nhận PNG, JPEG, GIF, WebP, SVG hoặc PDF; tối đa 10 MB), lưu trữ và tạo phần tử ảnh tại vị trí giữa vùng đang hiển thị trên bảng trắng.
5. NSD thấy ảnh xuất hiện trên bảng trắng của mình.
6. Hệ thống đồng bộ phần tử ảnh và hiển thị lên bảng trắng của tất cả thành viên khác trong phòng.

#### **Ngoại lệ:**

- Bước 4: File không thuộc loại được hỗ trợ → hệ thống báo "File không hợp lệ".
- Bước 4: File vượt quá 10 MB → hệ thống yêu cầu NSD chọn file nhỏ hơn.
- Bước 4: Hệ thống không lưu trữ được file → hệ thống thông báo lỗi và yêu cầu NSD thử lại sau.

#### **e. Kịch bản chia sẻ màn hình {#e.-kịch-bản-chia-sẻ-màn-hình}**

1. NSD đang ở trong phòng, muốn trình bày nội dung màn hình của mình cho các thành viên khác.
2. Hệ thống hiển thị nút Chia sẻ màn hình trên thanh công cụ.
3. NSD click nút Chia sẻ màn hình.
4. Hệ thống hiển thị hộp thoại cho phép chọn nguồn chia sẻ: toàn bộ màn hình, một cửa sổ ứng dụng hoặc một tab, kèm tùy chọn âm thanh hệ thống.
5. NSD chọn nguồn muốn chia sẻ và xác nhận.
6. Hệ thống bắt đầu truyền tải và hiển thị luồng màn hình (và âm thanh nếu có) dưới dạng lớp phủ trên bảng trắng của từng thành viên; các thành viên vẫn có thể vẽ hoặc ghi chú đè lên luồng màn hình.
7. NSD thay đổi độ phân giải (720p, 1080p, 4K) hoặc tốc độ khung hình (15, 30, 60 fps) nếu muốn.
8. Hệ thống cập nhật chất lượng luồng theo thông số NSD vừa chọn mà không cần ngắt luồng.
9. NSD click lại nút Chia sẻ màn hình (hoặc nút Dừng chia sẻ) để dừng.
10. Hệ thống ngắt luồng và gỡ lớp phủ khỏi bảng trắng của tất cả thành viên khác.

#### **Ngoại lệ:**

- Bước 5: NSD nhấn Hủy ở hộp thoại → hệ thống đóng hộp thoại, chức năng kết thúc và không có gì được chia sẻ.
- Bước 6: Băng thông mạng không đủ → hệ thống tự động giảm chất lượng luồng; nếu tiếp tục tệ, các thành viên có thể thấy hình ảnh giật hoặc mất tạm thời.

#### **f. Kịch bản gọi thoại (Voice Chat) {#f.-kịch-bản-gọi-thoại-(voice-chat)}**

1. NSD đang ở trong phòng, muốn trao đổi bằng giọng nói với các thành viên khác.
2. Hệ thống hiển thị nút Bật micro trên thanh công cụ.
3. NSD click nút Bật micro.

4. Hệ thống yêu cầu quyền truy cập microphone.
5. NSD cấp quyền truy cập microphone.
6. Hệ thống bắt đầu truyền tải giọng nói của NSD tới các thành viên khác trong phòng.
7. Các thành viên khác nghe thấy giọng của NSD trên thiết bị của họ.
8. NSD click lại nút Bật micro để tắt.
9. Hệ thống dừng truyền tải âm thanh; các thành viên khác không còn nghe thấy giọng của NSD.

#### **Ngoại lệ:**

- Bước 5: NSD từ chối cấp quyền microphone → hệ thống không kích hoạt chức năng và thông báo lỗi quyền truy cập.
- Bước 6: Thiết bị không có microphone hoặc microphone đang bị chiếm → hệ thống thông báo lỗi phần cứng.

#### **g. Kịch bản gửi tin nhắn chat {#g.-kịch-bản-gửi-tin-nhắn-chat}**

1. NSD đang ở trong phòng, muốn trao đổi bằng văn bản mà không làm gián đoạn không gian vẽ.
2. Hệ thống hiển thị nút mở khung Chat ở góc phòng.
3. NSD click nút mở khung Chat.
4. Hệ thống hiển thị khung Chat với danh sách tin nhắn trong phiên hiện tại và ô nhập tin nhắn ở cuối khung.
5. NSD nhập nội dung và nhấn Enter (hoặc nút Gửi).
6. Hệ thống hiển thị tin nhắn ngay trong khung Chat của NSD và gửi tới tất cả thành viên khác trong phòng.
7. Các thành viên khác thấy tin nhắn mới hiện trong khung Chat; nếu khung Chat đang đóng, hệ thống hiển thị thông báo ngắn và đánh dấu tin chưa đọc.

#### **Ngoại lệ:**

- Bước 5: Tin nhắn trống (chỉ chứa khoảng trắng) → hệ thống không gửi.
- Bước 6: Mất kết nối mạng → tin nhắn chỉ hiển thị cho NSD, không đến được các thành viên khác; sau khi kết nối lại, các tin nhắn cũ không được khôi phục.

#### **h. Kịch bản xuất và nhập bảng trắng {#h.-kịch-bản-xuất-và-nhập-bảng-trắng}**

1. NSD đang ở trong phòng và muốn lưu lại thành quả của phiên làm việc.
2. Hệ thống hiển thị bảng Cài đặt ở góc phòng.
3. NSD mở bảng Cài đặt và chọn mục Xuất file.
4. Hệ thống xuất toàn bộ nội dung bảng trắng hiện tại thành file JSON và tải xuống thiết bị của NSD.
5. NSD mở lại hệ thống ở lần khác và chọn Import từ file JSON.
6. Hệ thống hiển thị hộp thoại chọn file.
7. NSD chọn file JSON đã lưu trước đó.
8. Hệ thống khôi phục trạng thái bảng trắng từ file và đồng bộ nội dung tới tất cả thành viên trong phòng.

#### **Ngoại lệ:**

- Bước 3: Bảng trắng quá lớn → hệ thống báo "Không thể xuất file do dung lượng quá lớn".

- Bước 7: File JSON không hợp lệ (sai định dạng hoặc không phải do EvoDraw xuất ra) → hệ thống từ chối nhập và thông báo lỗi.

### **i. Kịch bản rời phòng {#i.-kịch-bản-rời-phòng}**

1. NSD đang ở trong giao diện phòng và muốn kết thúc phiên làm việc.
2. Hệ thống hiển thị nút Rời phòng trên giao diện phòng.
3. NSD click nút Rời phòng.
4. Hệ thống hiển thị hộp thoại xác nhận rời phòng.
5. NSD xác nhận rời phòng.
6. Hệ thống xóa NSD khỏi danh sách thành viên, thông báo tới các thành viên còn lại và chuyển NSD trở về trang chủ EvoDraw.

Ngoại lệ:

- Bước 5: NSD chọn Hủy ở hộp thoại xác nhận → hệ thống đóng hộp thoại, NSD tiếp tục ở trong phòng bình thường.
- Bước 6: Hệ thống không đồng bộ được trạng thái → hệ thống vẫn chuyển NSD về trang chủ và hiển thị cảnh báo rằng đồng bộ có thể chưa hoàn tất.

### **j. Kịch bản ghi chú đè lên màn hình chia sẻ bằng Desktop Overlay {#j.-kịch-bản-ghi-chú-đè-lên-màn-hình-chia-sẻ-bằng-desktop-overlay}**

1. NSD đang chia sẻ màn hình trong phòng và muốn vẽ ghi chú lên luồng màn hình để các thành viên khác thấy được.
2. Hệ thống hiển thị nút "Mở Desktop Overlay" trên giao diện chia sẻ màn hình.
3. NSD click nút đó.
4. Hệ thống khởi động ứng dụng Desktop Overlay và phủ một lớp cửa sổ trong suốt lên toàn màn hình của NSD.
5. Mặc định ứng dụng ở chế độ Làm việc: cửa sổ trong suốt, NSD vẫn thao tác bình thường với các ứng dụng bên dưới.
6. NSD nhấn **Ctrl+Shift+D** để chuyển sang chế độ Vẽ.
7. Hệ thống kích hoạt lớp vẽ; NSD nhấn giữ và kéo chuột để tạo nét ghi chú lên màn hình.
8. Hệ thống đồng bộ các nét ghi chú tới tất cả thành viên trong phòng.
9. Các thành viên khác thấy ghi chú hiện lên đè trên luồng màn hình chia sẻ của NSD gần như tức thời.
10. NSD nhấn lại **Ctrl+Shift+D** để chuyển về chế độ Làm việc; các nét ghi chú vẫn hiển thị đến khi NSD xóa hoặc dừng chia sẻ màn hình.

Ngoại lệ:

- Bước 4: Ứng dụng Desktop Overlay chưa được cài đặt → hệ thống không mở được ứng dụng; NSD cần tải về và cài đặt trước.
- Bước 8: Mất kết nối mạng → nét ghi chú hiển thị cục bộ trên Desktop nhưng không đồng bộ được tới các thành viên khác; khi kết nối lại, các nét đã vẽ không được gửi lại tự động.

### **3.1.2. Trích các lớp phân tích {#3.1.2.-trích-các-lớp-phân-tích}**

### **Mô tả hệ thống trong một đoạn văn:**

EvoDraw là một nền tảng không gian làm việc trực tuyến, cung cấp môi trường cộng tác bảng vẽ thời gian thực và giao tiếp đa phương tiện. Hệ thống cho phép người dùng tạo phòng mới (mã phòng và mã PIN do hệ thống tự sinh) hoặc tham gia vào phòng hiện có thông qua mã phòng và mã PIN. Trong phòng, người dùng có thể sử dụng các công cụ vẽ trên bảng trắng (bút vẽ, hình chữ nhật, hình tròn, đường thẳng, mũi tên, văn bản, tẩy), chèn hình ảnh, đồng bộ hóa nội dung bảng trắng với tất cả thành viên, cũng như nhìn thấy con trỏ chuột của thành viên khác theo thời gian thực, gọi thoại và trao đổi tin nhắn văn bản (chat) với các thành viên cùng phòng. Người dùng cũng có thể chia sẻ màn hình và cho phép các thành viên khác vẽ đè ghi chú lên luồng màn hình đó. Phòng không hoạt động trong 24 giờ sẽ tự động bị xóa để giải phóng tài nguyên.

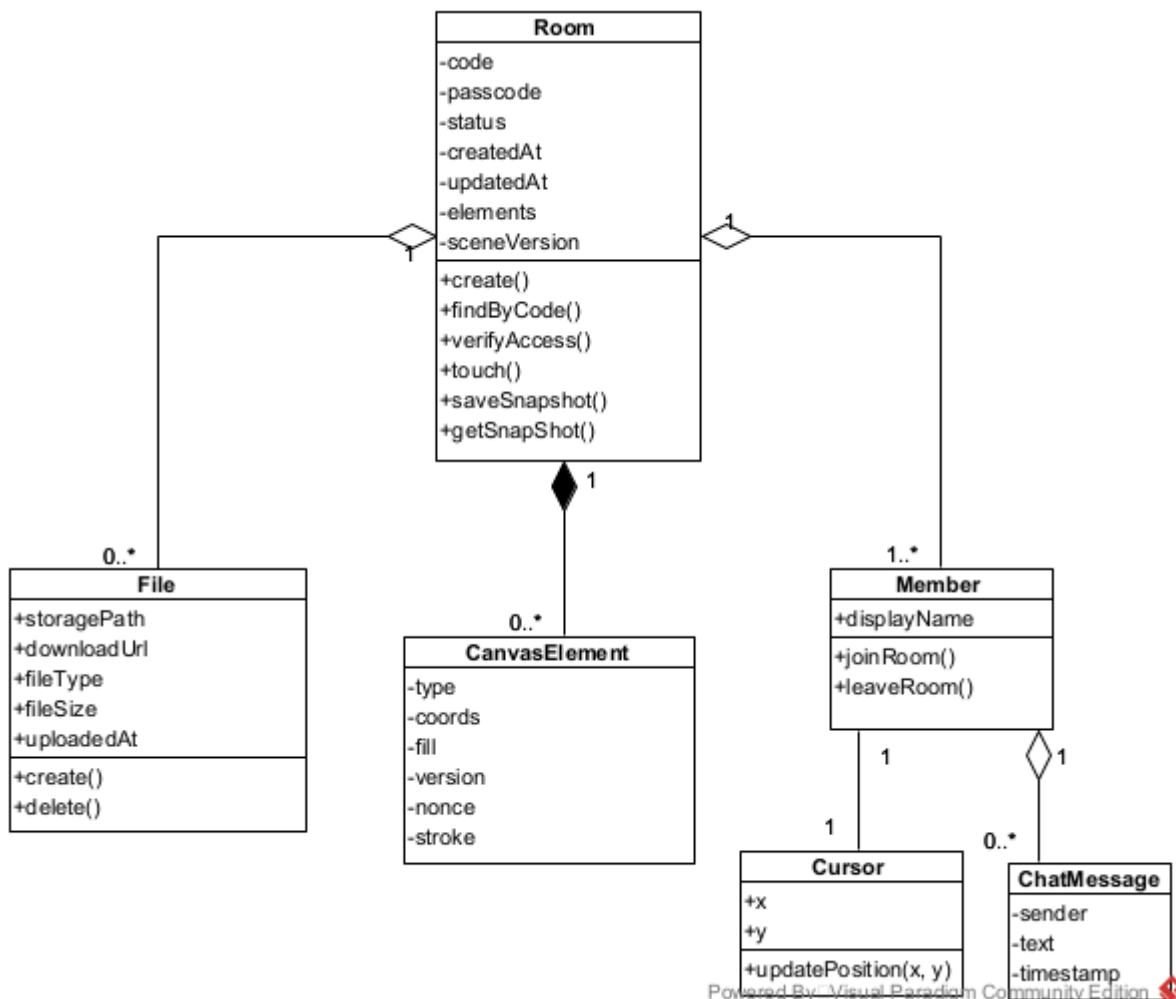
### **Phân tích các danh từ:**

- Hệ thống: danh từ chung chung → loại.
- Phòng: là đối tượng xử lý chính của hệ thống → là 1 lớp thực thể: Room
- Bảng vẽ: là thành phần giao diện → loại.
- Người dùng / Thành viên: tham gia phòng, có hành vi tương tác → là 1 lớp thực thể: Member
- Mã phòng, mã PIN: là thuộc tính của Room → loại.
- Công cụ vẽ: là thành phần giao diện → loại.
- Phần tử vẽ (đường vẽ, hình, văn bản): là đối tượng đồ họa trên bảng trắng → là 1 lớp thực thể: CanvasElement
- Hình ảnh / File: có thể được chèn vào bảng vẽ → là 1 lớp thực thể: File
- Con trỏ chuột: thể hiện vị trí của thành viên trong phòng → là 1 lớp thực thể: Cursor
- Tin nhắn chat: nội dung trao đổi giữa các thành viên → là 1 lớp thực thể: ChatMessage
- Gọi thoại / luồng âm thanh: truyền trực tiếp, không lưu trữ → loại.
- Luồng màn hình: truyền trực tiếp, không lưu trữ → loại.

=> Vậy chúng ta thu được các lớp thực thể là: Room, Member, CanvasElement, File, Cursor, ChatMessage.

### **Quan hệ giữa các lớp thực thể:**

- Một Room chứa nhiều Member tại một thời điểm, một Member chỉ thuộc một Room: quan hệ 1–n.
- Một Room chứa nhiều CanvasElement, mỗi CanvasElement thuộc duy nhất một Room: quan hệ 1–n (composition).
- Một Room có nhiều File, mỗi File chỉ thuộc một Room: quan hệ 1–n.
- Một Member có một Cursor tương ứng duy nhất tại mỗi thời điểm: quan hệ 1–1.
- Một Member có thể gửi nhiều ChatMessage trong phòng, mỗi ChatMessage được gửi bởi đúng một Member. Vậy quan hệ giữa Member và ChatMessage là 1 - n.
- Phòng không có hoạt động nào trong 24 giờ sẽ bị xóa. Khi đó, tất cả Member, CanvasElement, File, Cursor, ChatMessage liên quan đến phòng đó cũng không còn.



Hình 3.1:

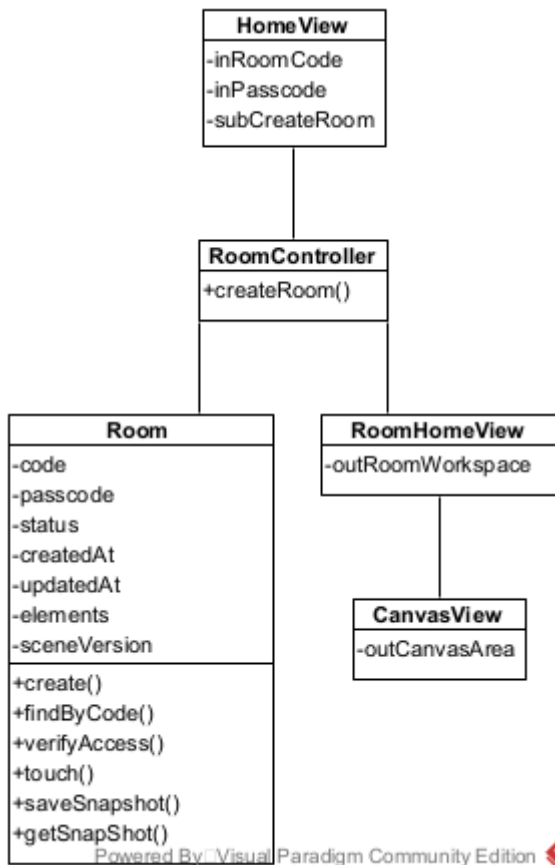
Biểu đồ lớp thực thể tổng quan của hệ thống EvoDraw {#hình-3.1:-biểu-đồ-lớp-thực-thể-tổng-quan-của-hệ-thống-evodraw}

### 3.1.3. Phân tích chi tiết từng module {#3.1.3.-phân-tích-chi-tiết-từng-module}

#### a. Chức năng Tạo phòng mới {#a.-chức-năng-tạo-phòng-mới}

##### Phân tích tĩnh — các lớp và thành phần cần thiết:

- Người dùng truy cập ứng dụng → giao diện trang chủ hiện lên → đề xuất lớp **LandingPage** (mới), có nút "Tạo phòng".
- Người dùng nhấn nút tạo phòng trên LandingPage → hệ thống cần chức năng createRoom() để điều phối việc tạo phòng → đề xuất lớp điều khiển **RoomController** (mới).
- RoomController yêu cầu khởi tạo một đối tượng phòng mới → sử dụng lớp thực thể **Room** đã có từ trích lớp thực thể. Hệ thống sử dụng hàm generateRoomCode() và generateRoomPassCode() (trong codeGenerator.js) để sinh mã phòng 6 ký tự và mã PIN 4 chữ số. Mã PIN được băm trực tiếp bằng bcrypt.hash() trong createRoomService() (không có hàm hashPasscode() riêng).
- Sau khi tạo phòng xong, hệ thống cần chuyển người dùng sang giao diện phòng → đề xuất lớp **RoomPage** (mới), là khung bao quanh phòng làm việc.
- RoomPage cần hiển thị vùng bảng trắng → đề xuất lớp **Canvas** (mới). (đã sửa: CanvasView → Canvas, component thực tế: Canvas.jsx)

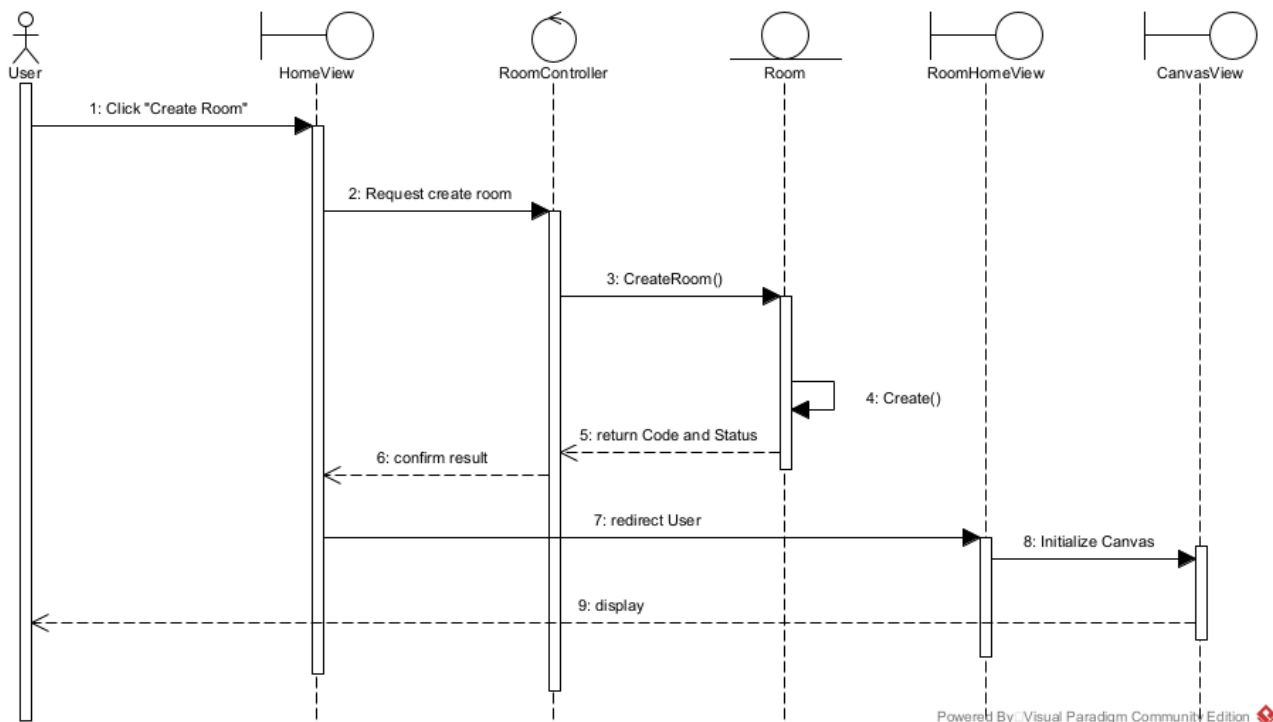


Hình 3.2: Biểu đồ các lớp phân tích — chức năng Tạo phòng mới {#hình-3.2:-biểu-

đồ-các-lớp-phân-tích—chức-năng-tạo-phòng-mới}

### Phân tích động — biểu đồ tuần tự:

1. Người dùng click nút "Tạo phòng" trên LandingPage.
2. LandingPage gọi RoomController xử lý.
3. RoomController khởi tạo một đối tượng Room mới.
4. createRoomService() khởi tạo phòng mới trong CSDL.
5. Room trả kết quả (code, passcode plaintext) cho RoomController.
6. RoomController trả kết quả cho LandingPage.
7. LandingPage gọi RoomPage.
8. RoomPage khởi tạo Canvas
9. Canvas hiển thị cho người dùng.



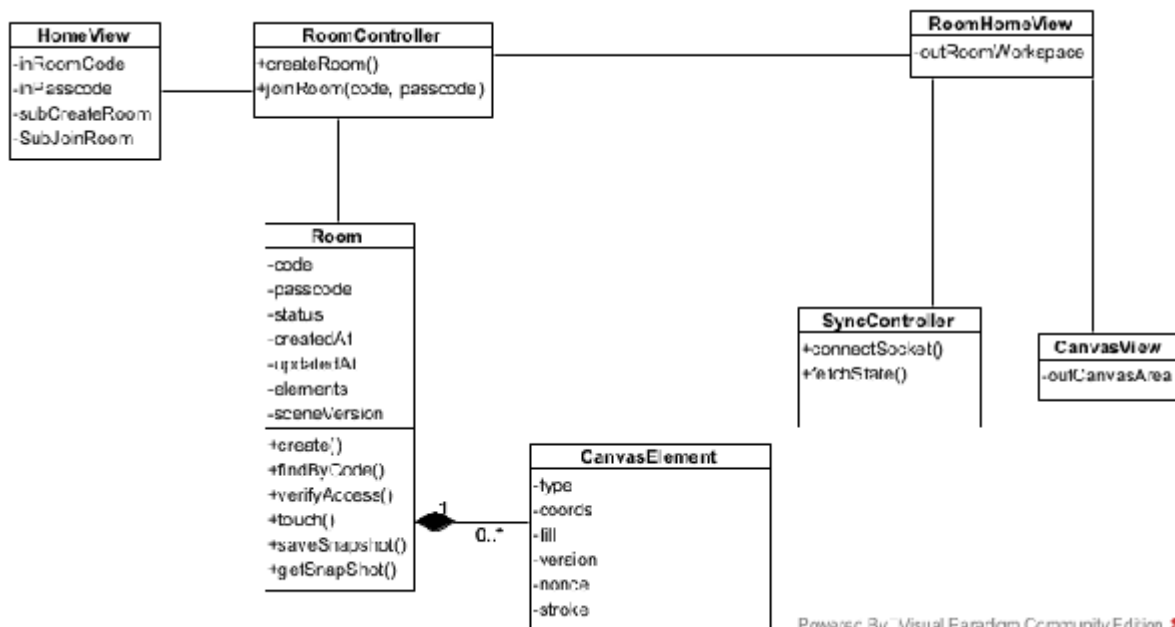
Hình 3.3: Biểu đồ tuần tự — chức năng Tạo phòng mới {#hình-3.3:-biểu-đồ-tuần-tự---chức-năng-tạo-phòng-mới}

## b. Chức năng Tham gia phòng {#b.-chức-năng-tham-gia-phòng}

### Phân tích tĩnh — các lớp và thành phần cần thiết:

- Người dùng ở trang chủ → nhập mã phòng và mã PIN vào lớp **LandingPage** đã có, form đã có thêm nút "Tham gia".
- Nhấn "Tham gia" → hệ thống cần chức năng joinRoom(code, pin) → là hành động của lớp **RoomController** đã có.
- RoomController tra cứu và xác thực phòng thông qua lớp **Room** đã có. Phòng được tra cứu qua getRoom({ code }) trong room.service.js. PIN được so khớp bằng bcrypt.compare() trực tiếp trong room.handler.js. Thời gian tồn tại phòng được gia hạn tự động qua Mongoose timestamps (không có hàm touch() riêng).
- Xác thực thành công, hệ thống chuyển người dùng sang lớp **RoomPage** đã có để hiển thị phòng.
- RoomPage cần mở kết nối thời gian thực để tải nội dung bảng trắng hiện có → đề xuất lớp điều khiển **SyncController** (mới), quản lý đồng bộ thời gian thực giữa các máy.
- Nội dung bảng trắng được tải về và hiển thị trên **Canvas** đã có.





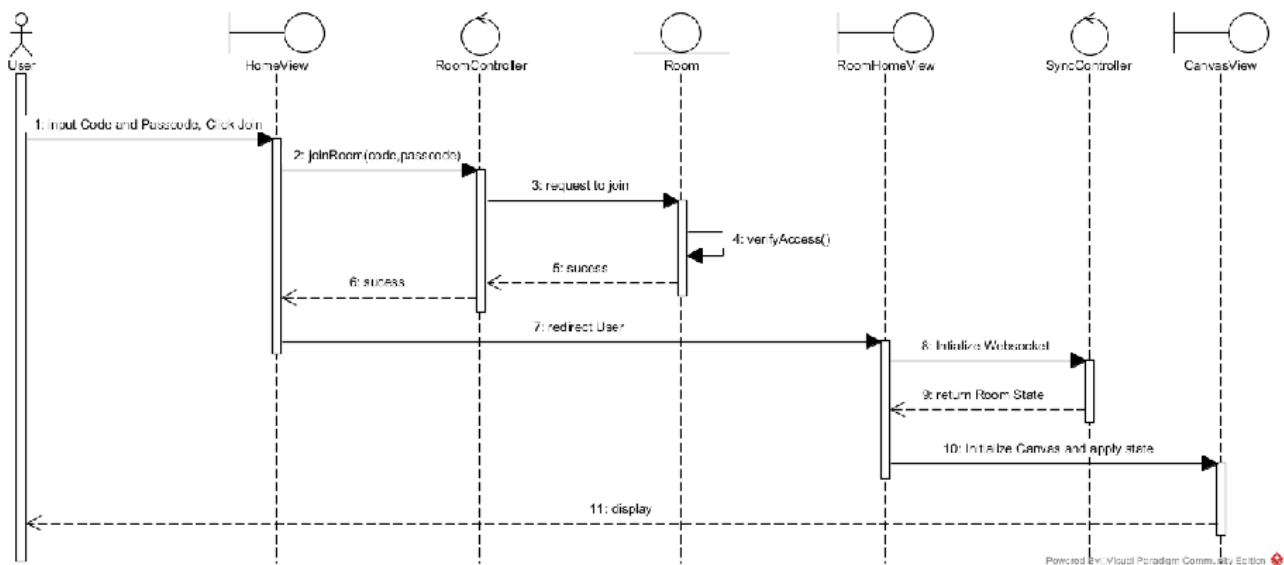
Powered By Visual Paradigm Community Edition

Hình 3.4:

Biểu đồ các lớp phân tích — chức năng Tham gia phòng {#**hình-3.4:-biểu-đồ-các-lớp-phân-tích—chức-năng-tham-gia-phòng**}

### Phân tích động — biểu đồ tuần tự:

1. Người dùng nhập mã phòng, mã PIN vào LandingPage và click "Tham gia".
2. LandingPage gọi RoomController.joinRoom(code, pin).
3. RoomController gọi getRoom({ code }) trong room.service.js để tra cứu phòng.
4. Room trả đối tượng phòng cho RoomController.
5. RoomController dùng bcrypt.compare(pin, room.passcode) để kiểm tra mã PIN.
6. Nếu xác thực sai, Room trả lỗi cho RoomController. RoomController trả lỗi cho LandingPage.  
LandingPage hiển thị thông báo "Mã phòng hoặc PIN không hợp lệ" cho người dùng.
7. Nếu xác thực đúng, Mongoose tự cập nhật updatedAt khi room được lưu để gia hạn thời gian tồn tại, và trả kết quả thành công.
8. RoomController trả kết quả cho LandingPage.
9. LandingPage gọi RoomPage.
10. RoomPage gọi SyncController yêu cầu mở kết nối thời gian thực.
11. SyncController kết nối tới Server, nhận về trạng thái bảng trắng hiện tại, rồi trả dữ liệu cho RoomPage.
12. RoomPage khởi tạo Canvas và hiển thị nội dung bảng trắng cho người dùng.

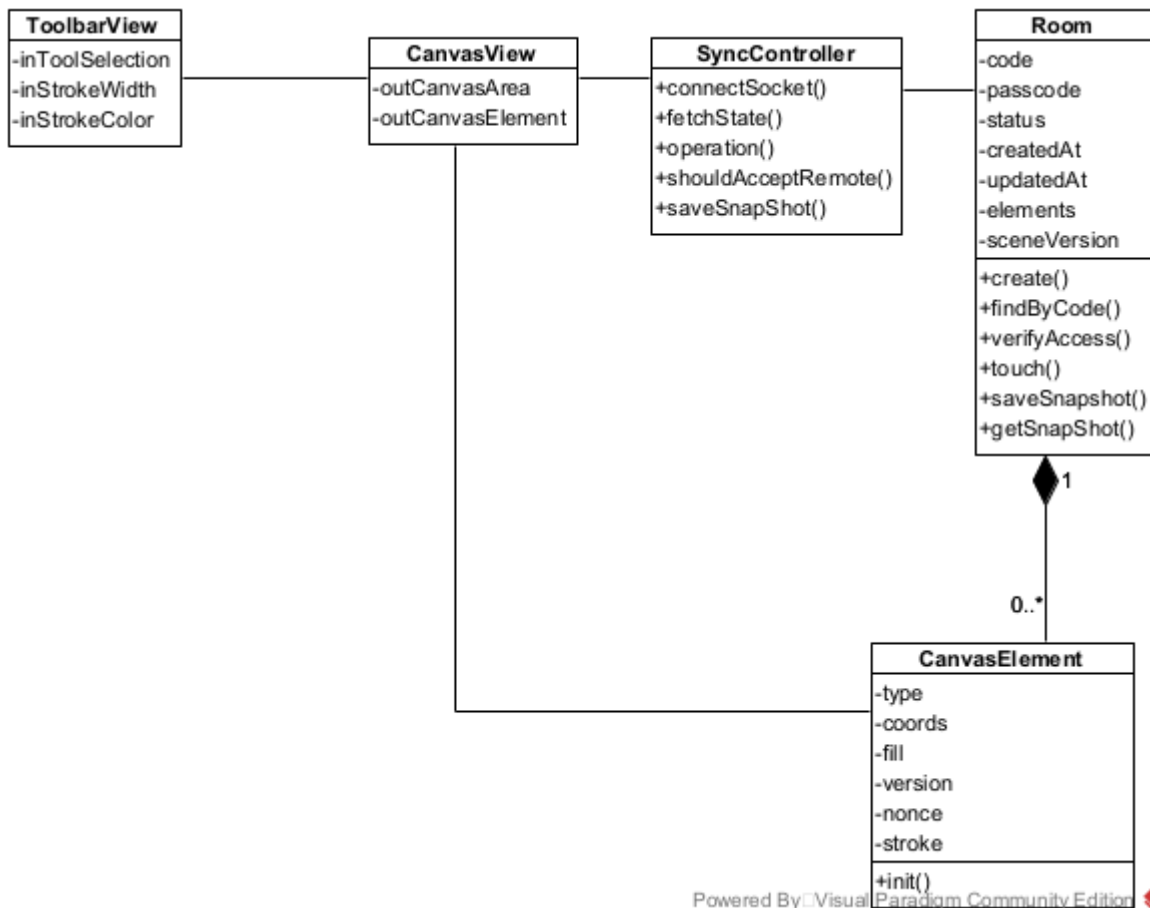


Hình 3.5: Biểu đồ tuần tự — chức năng Tham gia phòng {#hình-3.5:-biểu-đồ-tuần-tự---chức-năng-tham-gia-phòng}

### c. Chức năng Vẽ tự do trên bảng trắng {#c.-chức-năng-vẽ-tự-do-trên-bảng-trắng}

#### Phân tích tĩnh — các lớp và thành phần cần thiết:

- Vùng bảng trắng là lớp **Canvas** đã có. Ngoài ra, người dùng cần một thanh công cụ để chọn bút/tẩy/hình dạng và tùy chỉnh nét → đề xuất lớp **Toolbar** (mới). (đã sửa: *CanvasView* → *Canvas*, *ToolbarView* → *Toolbar*, *component* thực tế: *Canvas.jsx* / *Toolbar.jsx*)
- Người dùng chọn công cụ trên Toolbar → Toolbar cần tác động lên Canvas để cấu hình công cụ vẽ. Lớp Canvas cần bổ sung chức năng *setTool()* để chọn công cụ hiện hành và *setStyle()* để cấu hình màu/độ dày/kiểu nét.
- Người dùng vẽ trên Canvas → mỗi nét vẽ được biểu diễn bằng một đối tượng **CanvasElement** đã có (nhúng trong *Room.elements*).
- Hệ thống cần đồng bộ phần tử vẽ cho các thành viên khác → sử dụng lớp **SyncController** đã có. Lớp *SyncController* cần bổ sung chức năng *broadcastOperation()* để phát phần tử vẽ ra mạng, *shouldAcceptRemote()* để giải xung đột LWW khi nhận phần tử từ máy khác, và *saveSnapshot()* để định kỳ phát sự kiện *save\_snapshot* qua *Socket.IO*, kích hoạt *updateRoomService()* trên server để lưu snapshot vào CSDL.

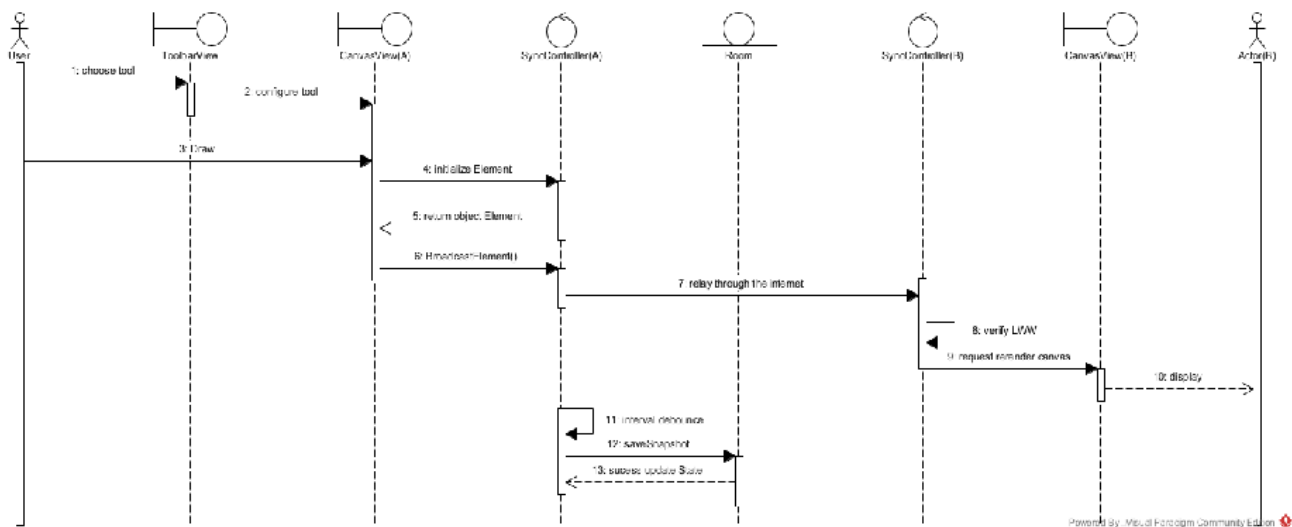


Hình 3.6:

Biểu đồ các lớp phân tích — chức năng Vẽ tự do trên bảng trắng {#hình-3.6:-biểu-đồ-các-lớp-phân-tích——chức-năng-vẽ-tự-do-trên-bảng-trắng}

### Phân tích động — biểu đồ tuần tự:

1. Người dùng A chọn công cụ vẽ, màu và độ dày trên Toolbar.
2. Toolbar gọi Canvas.setTool() và Canvas.setStyle() để cấu hình công cụ vẽ.
3. Người dùng A thực hiện thao tác vẽ trên Canvas.
4. Canvas khởi tạo đối tượng CanvasElement chứa thông tin nét vẽ mới.
5. Canvas gọi SyncController(A).broadcastOperation(CanvasElement) để phát phần tử ra mạng.
6. SyncController (Máy A) chuyển tiếp CanvasElement tới SyncController (Máy B).
7. SyncController (Máy B) gọi shouldAcceptRemote() để xử lý xung đột phiên bản.
8. Nếu chấp nhận phần tử, SyncController (Máy B) gọi Canvas (Máy B) để cập nhật nét vẽ.
9. Canvas (Máy B) hiển thị nét vẽ mới cho Người dùng B.
10. Định kỳ, SyncController (Máy A) phát sự kiện save\_snapshot qua Socket.IO để lưu trạng thái bảng trắng vào CSDL thông qua updateRoomService().

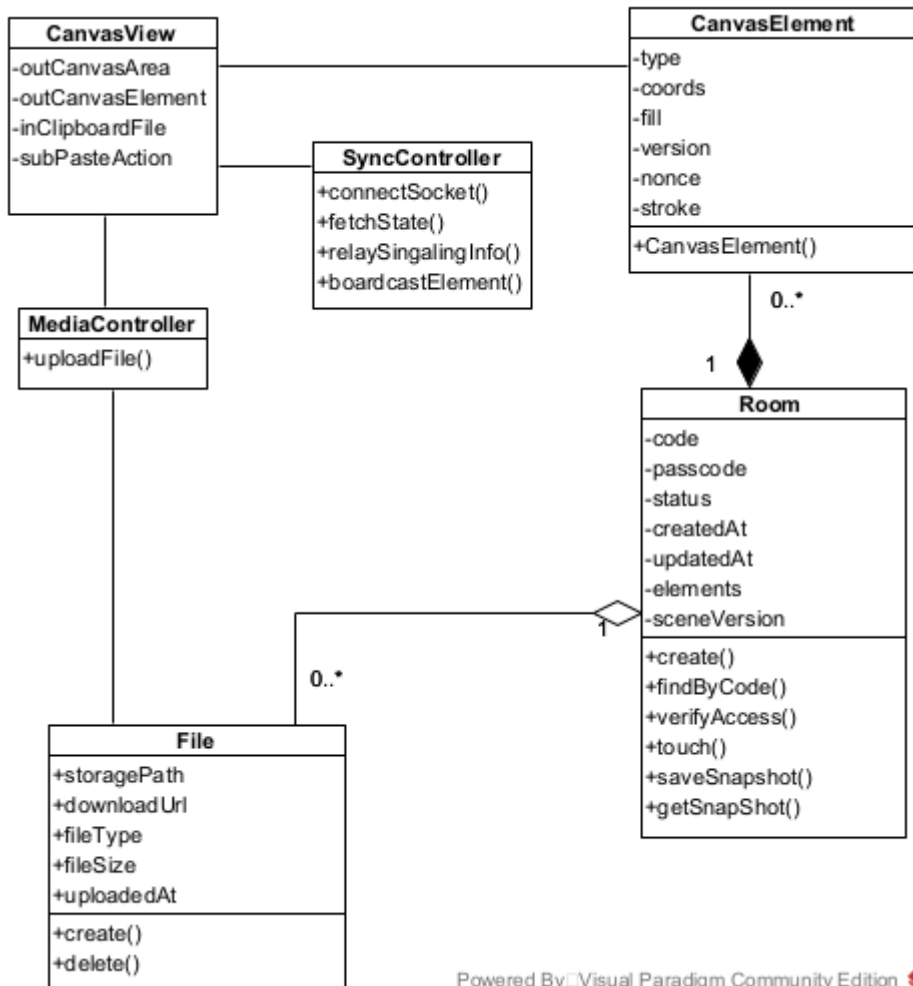


Hình 3.7: Biểu đồ tuần tự — chức năng Vẽ tự do trên bảng trắng {#hình-3.7:-biểu-đồ-tuần-tự—chức-năng-vẽ-tự-do-trên-bảng-trắng}

#### d. Chức năng Chèn ảnh vào bảng trắng {#d.-chức-năng-chèn-ảnh-vào-bảng-trắng}

##### Phân tích tĩnh — các lớp và thành phần cần thiết:

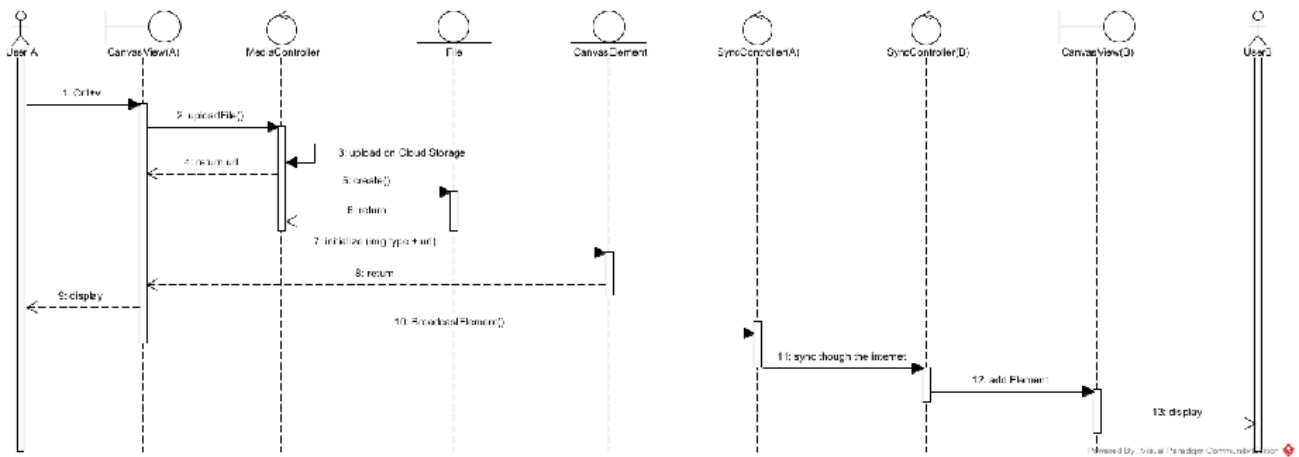
- Người dùng dán ảnh (Ctrl+V) trực tiếp vào **Canvas** đã có
- Hệ thống cần tải file ảnh lên kho lưu trữ đám mây (hệ thống ngoài) và quản lý thông tin tham chiếu → đề xuất lớp điều khiển **MediaController** (mới), có chức năng uploadFile(file) để tải file lên kho lưu trữ và trả về URL công khai.
- MediaController lưu thông tin tham chiếu (fileId, roomId, mimeType, dataURL, size) → sử dụng lớp thực thể **File** đã có. Lớp File cần bổ sung chức năng create() để tạo một bản ghi metadata mới trong CSDL.
- Sau khi có URL, Canvas khởi tạo một đối tượng **CanvasElement** đã có (loại ảnh) để hiển thị trên bảng trắng.
- Phần tử ảnh được đồng bộ tới các thành viên khác → sử dụng **SyncController** đã có (broadcastOperation()).



Hình 3.8: Biểu đồ các lớp phân tích — chức năng Chèn ảnh vào bảng trắng {#**hình-3.8:-biểu-đồ-các-lớp-phân-tích**—**chức-năng-chèn-ảnh-vào-bảng-trắng**}

### Phân tích động — biểu đồ tuần tự:

1. Người dùng A dán ảnh (Ctrl+V) hoặc chọn file ảnh để tải lên trên Canvas.
2. Canvas gọi MediaController.uploadFile(file).
3. MediaController trả về URL công khai.
4. MediaController gọi File.create() để lưu metadata (fileId, roomId, mimeType, dataURL, size) vào CSDL.
5. File trả kết quả cho MediaController.
6. MediaController trả URL cho Canvas.
7. Canvas khởi tạo đối tượng CanvasElement (loại ảnh) kèm URL.
8. CanvasElement trả về đối tượng bức ảnh
9. Canvas hiển thị ảnh cho người dùng A
10. Canvas gọi SyncController(A).broadcastOperation(CanvasElement).
11. SyncController (Máy A) chuyển tiếp CanvasElement tới SyncController (Máy B).
12. SyncController (Máy B) gọi Canvas (Máy B) để thêm phần tử ảnh.
13. Canvas (Máy B) tải ảnh từ URL công khai và hiển thị cho Người dùng B.

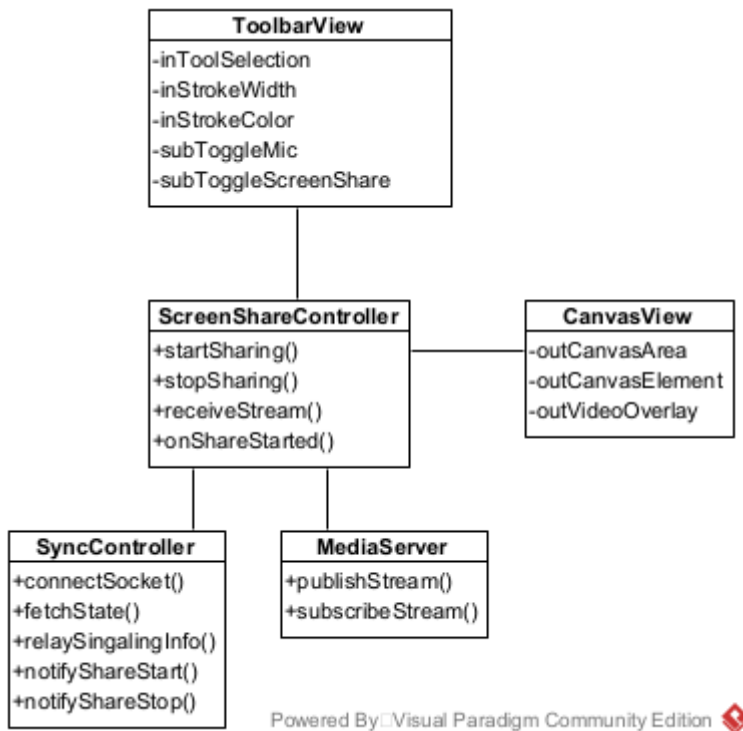


Hình 3.9: Biểu đồ tuần tự — chức năng Chèn ảnh vào bảng trắng {#**hình-3.9:-biểu-đồ-tuần-tự-—chức-năng-chèn-ảnh-vào-bảng-trắng**}

### e. Chức năng Chia sẻ màn hình {#**e.-chức-năng-chia-sẻ-màn-hình**}

#### Phân tích tĩnh — các lớp và thành phần cần thiết:

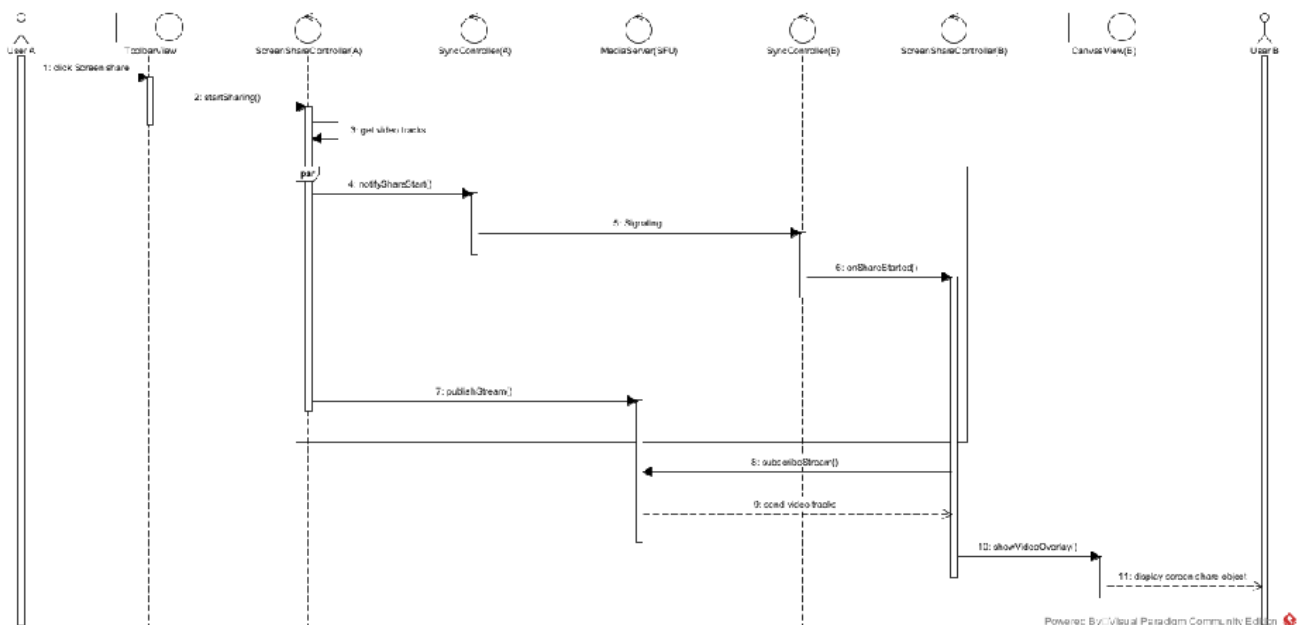
- Người trình bày thực hiện thao tác bật/tắt chia sẻ từ **Toolbar** đã có.
- Hệ thống cần một đối tượng điều khiển quản lý việc yêu cầu quyền truy cập màn hình, cũng như đóng gói và xử lý luồng media → đề xuất lớp điều khiển **StreamController** (mới), có các chức năng startSharing() để bắt đầu, stopSharing() để dừng và onShareStarted() để xử lý khi nhận thông báo chia sẻ từ thành viên khác.
- Việc trao đổi các tín hiệu điều khiển (Signaling) để thông báo trạng thái chia sẻ được thực hiện thông qua lớp **SyncController** đã có.
- Nhằm đảm bảo hiệu suất mạng, luồng dữ liệu media không truyền trực tiếp giữa các Client mà được định tuyến tập trung qua một máy chủ SFU → đề xuất lớp **MediaServer** (ngoại vi/điều khiển luồng).
- Ở phía người xem, luồng video được hiển thị dưới dạng lớp phủ trên **Canvas** đã có (người xem vẫn có thể vẽ đè lên luồng video).



Hình 3.10: Biểu đồ các lớp phân tích — chức năng Chia sẻ màn hình (#hình-3.10:-biểu-đồ-các-lớp-phân-tích——chức-năng-chia-sẻ-màn-hình)

### Phân tích động — biểu đồ tuần tự:

1. Người trình bày (Máy A) click nút "Chia sẻ màn hình" trên Toolbar.
2. Toolbar gọi StreamController(A).startSharing().
3. StreamController yêu cầu thiết bị cấp quyền và lấy luồng video màn hình.
4. StreamController (A) gọi SyncController (A) để phát tín hiệu (Signaling) báo phòng biết có người bắt đầu chia sẻ.
5. Đồng thời, StreamController (A) kết nối và đẩy luồng media (Publish) lên MediaServer.
6. SyncController (A) chuyển tiếp tín hiệu qua mạng tới SyncController (Máy B).
7. Khi nhận được thông báo, SyncController (B) gọi StreamController(B).onShareStarted() để chuẩn bị nhận dữ liệu.
8. StreamController (B) tiến hành kết nối (Subscribe) tới MediaServer
9. MediaServer gửi luồng video về cho StreamController(B).
10. StreamController (B) gọi Canvas để hiển thị luồng video dưới dạng lớp phủ.
11. Canvas hiển thị luồng video cho Người dùng B.

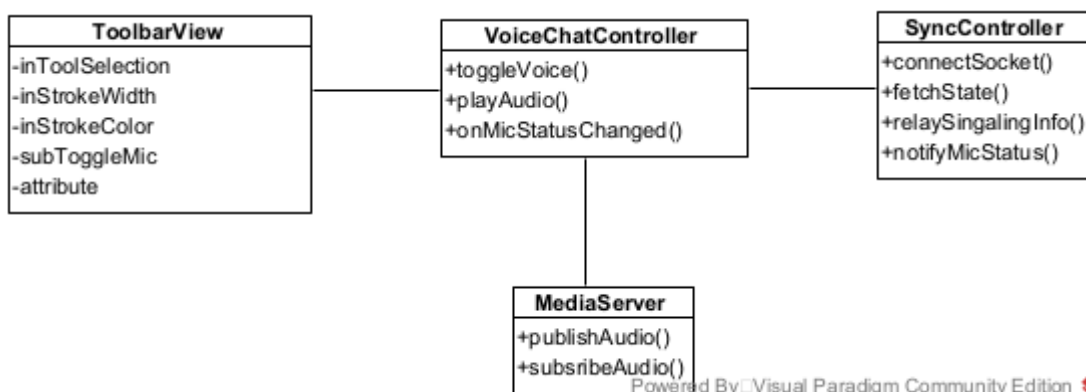


Hình 3.11: Biểu đồ tuần tự — chức năng Chia sẻ màn hình {#hình-3.11:-biểu-đồ-tuần-tự——chức-năng-chia-sẻ-màn-hình}

## f. Chức năng Gọi thoại (Voice Chat) {#f.-chức-năng-gọi-thoại-(voice-chat)}

### Phân tích tĩnh — các lớp và thành phần cần thiết:

- Người dùng bật/tắt micro bằng thao tác tại lớp **Toolbar** đã có.
- Hệ thống cần một đối tượng điều khiển quản lý việc lấy luồng âm thanh từ thiết bị và giao tiếp với máy chủ → đề xuất lớp điều khiển **StreamController** (mới), có chức năng toggleVoice() để bật/tắt micro. Việc phát âm thanh và cập nhật trạng thái micro của các thành viên khác được LiveKit SDK xử lý tự động qua ParticipantEvent (không có hàm playAudio() hay onMicStatusChanged() riêng).
- Việc trao đổi tín hiệu (Signaling) để cập nhật trạng thái micro (ai đang nói, ai đang tắt tiếng) được thực hiện thông qua lớp **SyncController** đã có.
- Tương tự như chia sẻ màn hình, luồng âm thanh được định tuyến tập trung qua máy chủ SFU nhằm giảm tải băng thông cho client → lớp **MediaServer**.



Hình 3.12: Biểu đồ

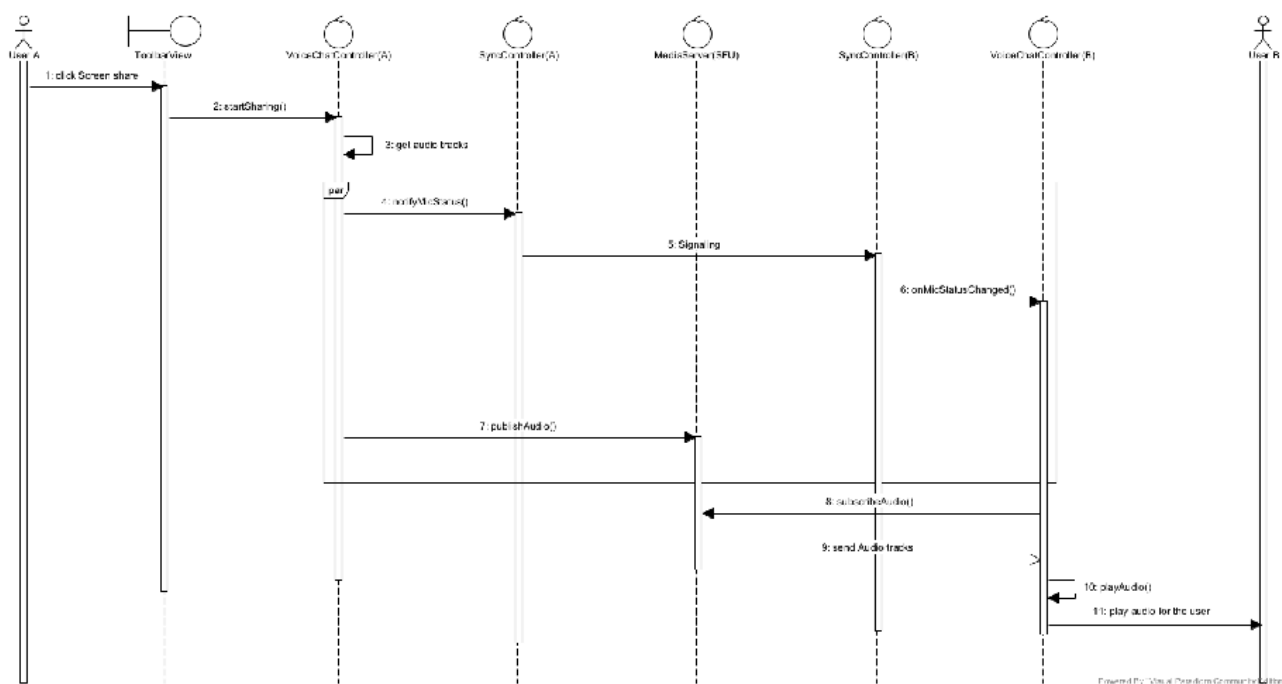
các lớp phân tích — chức năng Gọi thoại (Voice Chat) {#hình-3.12:-biểu-đồ-các-lớp-phân-tích——chức-năng-gọi-thoại-(voice-chat)}

### Phân tích động — biểu đồ tuần tự:

1. Người dùng A click nút "Bật micro" trên Toolbar.



2. Toolbar gọi `StreamController(A).toggleVoice()`.
3. `StreamController` yêu cầu cấp quyền và thu thập luồng âm thanh từ micro thiết bị.
4. `StreamController` gọi `SyncController` để phát tín hiệu cập nhật trạng thái micro (VD: trạng thái đang nói).
5. Đồng thời, `StreamController (A)` kết nối và đẩy luồng âm thanh (`Publish`) lên `MediaServer`.
6. `SyncController (A)` chuyển tiếp tín hiệu qua mạng tới `SyncController (Máy B)`.
7. `LiveKit SDK` tự động cập nhật trạng thái micro của participant cho `StreamController(B)` qua `ParticipantEvent`.
8. `StreamController (B)` duy trì kết nối (`Subscribe`) tới `MediaServer` để nhận luồng âm thanh của Máy A.
9. `LiveKit SDK` tự động phát luồng âm thanh ra loa thiết bị của Người dùng B. (đã sửa: xóa `playAudio()` — *LiveKit tự xử lý, không cần gọi thủ công*)
10. Người dùng B nghe thấy giọng nói của Người dùng A.

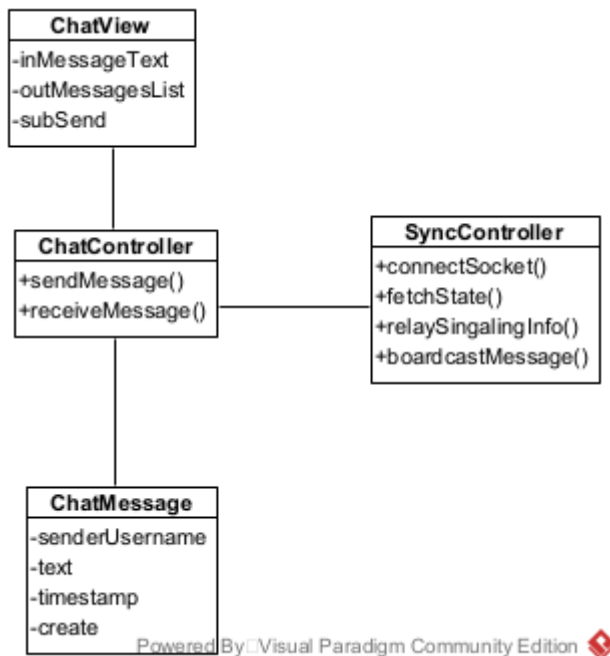


Hình 3.13: Biểu đồ tuần tự — chức năng Gọi thoại (Voice Chat) {#hình-3.13:-biểu-đồ-tuần-tự-—chức-năng-gọi-thoại-(voice-chat)}

## g. Chức năng Gửi tin nhắn chat {#g.-chức-năng-gửi-tin-nhắn-chat}

### Phân tích tĩnh — các lớp và thành phần cần thiết:

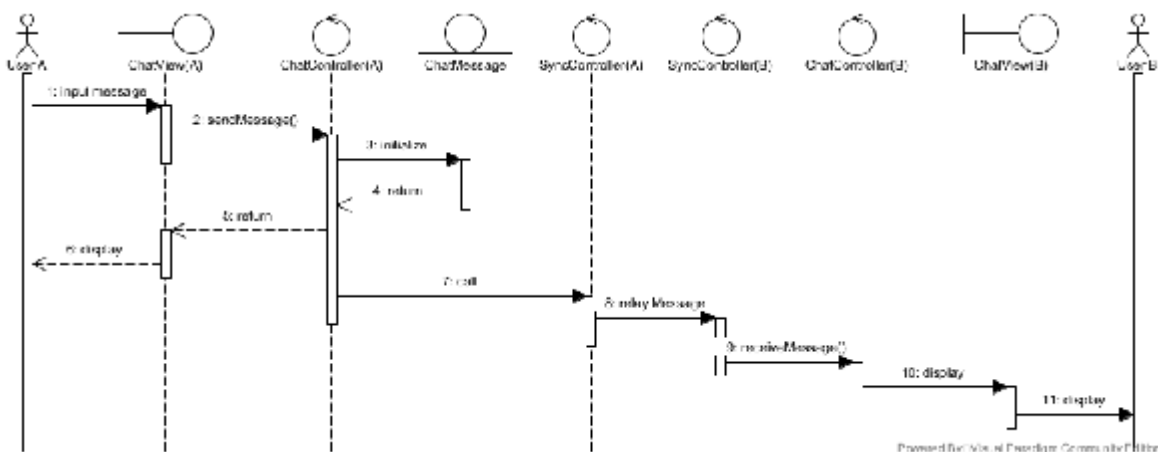
- Hệ thống cần một khung trao đổi tin nhắn có thể đóng/mở trong phòng → đề xuất lớp **ChatPanel** (mới) quản lý việc nhập và hiển thị tin nhắn. (đã sửa: `ChatView` → `ChatPanel`, component thực tế: `ChatPanel.jsx`)
- Người dùng gửi tin nhắn trên `ChatPanel` → hệ thống cần một đối tượng điều khiển chịu trách nhiệm phát tin nhắn đi và tiếp nhận tin nhắn đến → đề xuất lớp điều khiển **ChatController** (mới), có chức năng `sendMessage(text)` để phát tin nhắn.
- Mỗi tin nhắn được biểu diễn bằng một đối tượng **ChatMessage** đã có.
- Việc phát tin nhắn tới các thành viên khác trong phòng được thực hiện thông qua lớp **SyncController** đã có.



Hình 3.14: Biểu đồ các lớp phân tích — chức năng Gửi tin nhắn chat {#hình-3.14:-biểu-đồ-các-lớp-phân-tích-—chức-năng-gửi-tin-nhắn-chat}

### Phân tích động — biểu đồ tuần tự:

1. Người dùng A nhập nội dung tin nhắn vào ChatPanel và click gửi.
2. ChatPanel gọi ChatController(A).sendMessage(text).
3. ChatController (Máy A) khởi tạo đối tượng ChatMessage (sender, text, timestamp).
4. ChatController (Máy A) gọi ChatPanel hiển thị tin nhắn cho Người dùng A (phản hồi tức thì).
5. ChatController (Máy A) gọi SyncController(A) để phát ChatMessage ra mạng.
6. SyncController (Máy A) chuyển ChatMessage tới SyncController (Máy B).
7. SyncController (Máy B) gọi ChatController (Máy B) với ChatMessage nhận được.
8. ChatController (Máy B) gọi ChatPanel (Máy B) hiển thị tin nhắn cho Người dùng B. Nếu ChatPanel đang đóng, ChatPanel hiển thị toast thông báo và tăng badge tin chưa đọc.

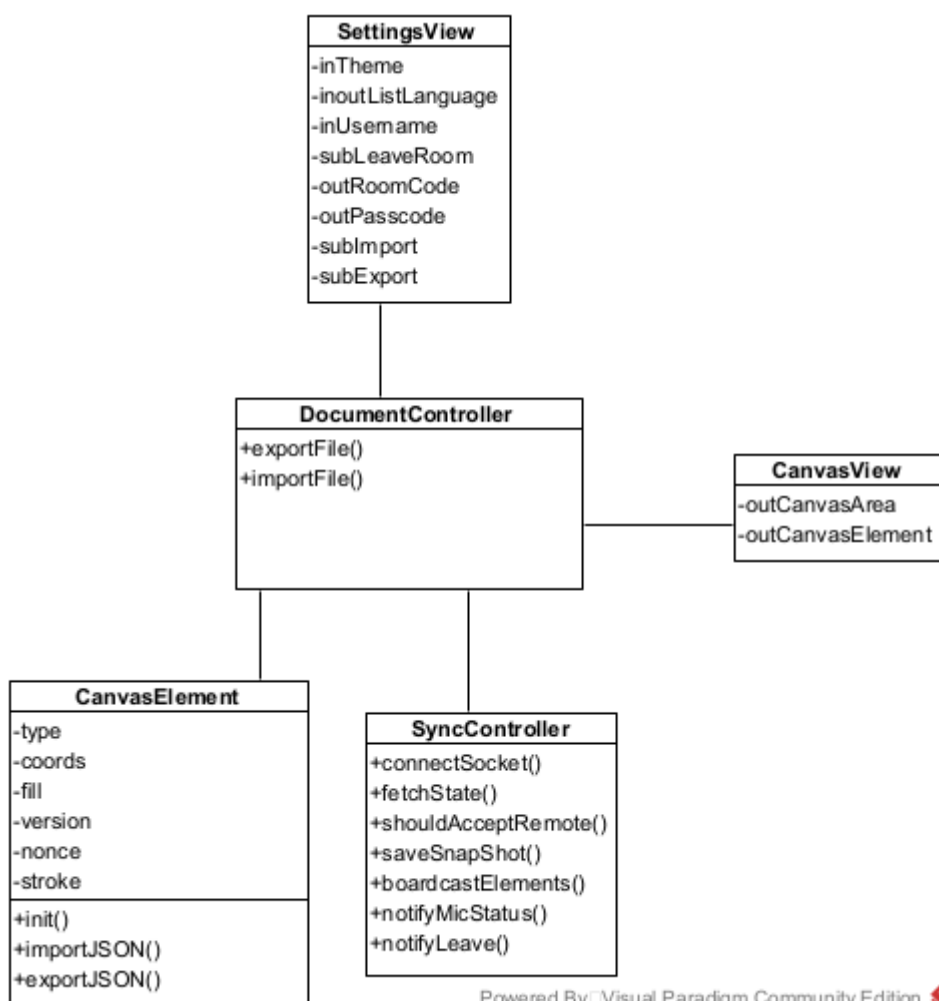


Hình 3.15: Biểu đồ tuần tự — chức năng Gửi tin nhắn chat {#hình-3.15:-biểu-đồ-tuần-tự-—chức-năng-gửi-tin-nhắn-chat}

### h. Chức năng Nhập và xuất file bảng trắng {#h.-chức-năng-nhập-và-xuất-file-bảng-trắng}

## Phân tích tĩnh — các lớp và thành phần cần thiết

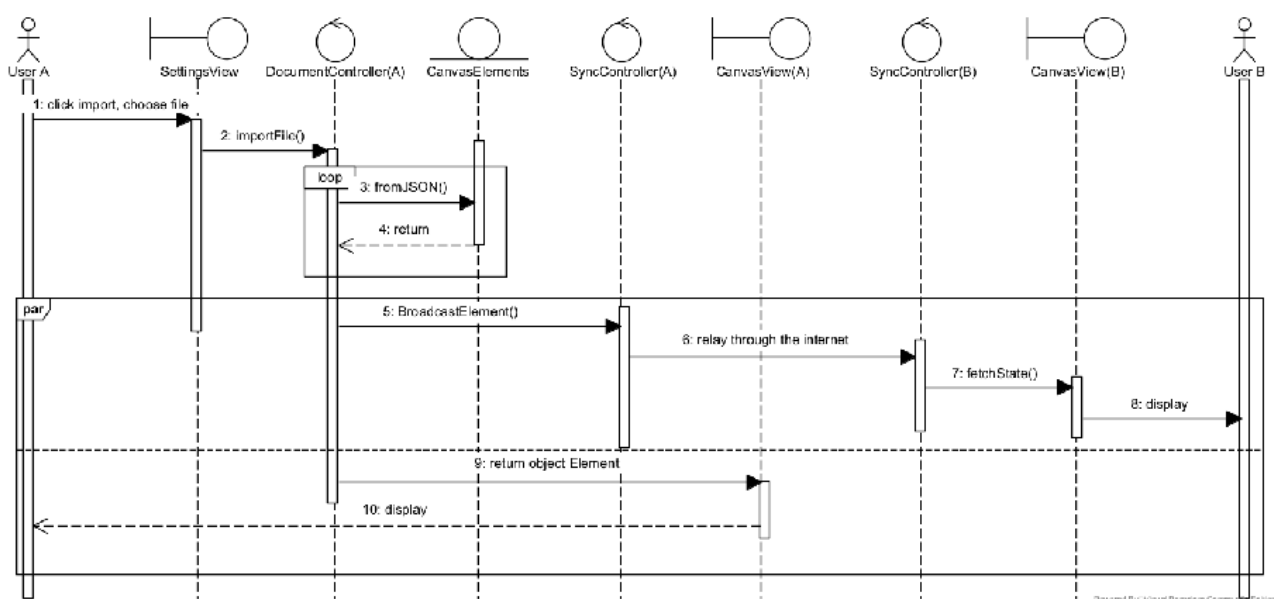
- Hệ thống cần một menu cấu hình của phòng (đổi theme, đổi nền canvas, nhập/xuất file, rời phòng...) → đề xuất lớp **SettingsPanel** (mới), trong đó có nút Nhập file và nút Xuất file.
- Cả hai thao tác nhập và xuất đều liên quan đến việc tuần tự hóa và giải tuần tự hóa dữ liệu bảng trắng → đề xuất lớp điều khiển **DocumentController** (mới), tập trung xử lý tài liệu với hai chức năng: `importFile(file)` để đọc file JSON và khôi phục bảng trắng, và `exportFile()` để tuần tự hóa bảng trắng thành file JSON tải xuống.
- Nhập file:** Người dùng chọn file JSON từ thiết bị trên SettingsPanel → SettingsPanel gọi `DocumentController.importFile(file)`. DocumentController phân tích cú pháp file JSON và tái tạo các phần tử vẽ thông qua lớp **CanvasElement** đã có. Quá trình giải tuần tự hóa được thực hiện bởi `loadCanvasSnapshot(canvas, snapshot, state)` trong `canvasSerializer.js`. Sau khi tái tạo, DocumentController hiển thị các phần tử trên Canvas và gọi **SyncController** đã có để đồng bộ tới các thành viên khác.
- Xuất file:** Người dùng chọn Xuất file trên SettingsPanel → SettingsPanel gọi `DocumentController.exportFile()`. DocumentController gọi `serializeCanvas(canvas)` trong `canvasSerializer.js` để tuần tự hóa tất cả phần tử vẽ thành JSON, đóng gói thành file và trả về để kích hoạt tải xuống.



Hình 3.16: Biểu đồ các lớp phân tích — chức năng Nhập và xuất file bảng trắng {#**hình-3.16:-biểu-đồ-các-lớp-phân-tích-—chức-năng-nhập-và-xuất-file-bảng-trắng**}

## Phân tích động — biểu đồ tuần tự (Nhập file):

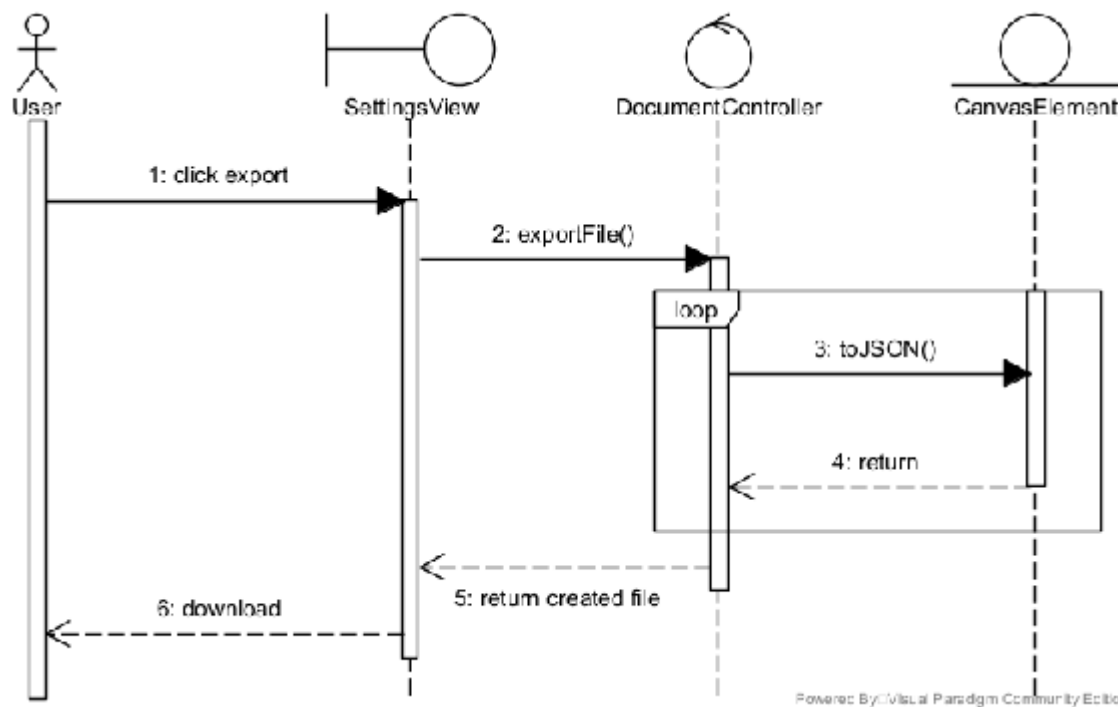
1. Người dùng A click "Nhập file" trên SettingsPanel và chọn file JSON từ thiết bị.
2. SettingsPanel gọi DocumentController.importFile(file).
3. DocumentController gọi loadCanvasSnapshot(canvas, snapshot, state) trong canvasSerializer.js để tái tạo các phần tử.
4. CanvasElement trả đối tượng đã khởi tạo cho DocumentController.
5. DocumentController thực hiện đồng thời hai việc:
  - **Hiển thị:** DocumentController đưa các phần tử lên Canvas. Canvas hiển thị kết quả cho Người dùng A (phản hồi tức thì).
  - **Đồng bộ:** DocumentController gọi SyncController(A).broadcastOperation() để phát các phần tử mới ra mạng.
6. SyncController (Máy A) chuyển tiếp các phần tử tới SyncController (Máy B).
7. SyncController (Máy B) cập nhật bảng trắng của Người dùng B.



Hình 3.17: Biểu đồ tuần tự — chức năng Nhập file bảng trắng {#hình-3.17:-biểu-đồ-tuần-tự---chức-năng-nhập-file-bảng-trắng}

## Phân tích động — biểu đồ tuần tự (Xuất file):

1. Người dùng click "Xuất file" trên SettingsPanel.
2. SettingsPanel gọi DocumentController.exportFile().
3. DocumentController gọi serializeCanvas(canvas) trong canvasSerializer.js để tuần tự hóa tất cả phần tử về thành JSON.
4. CanvasElement trả dữ liệu JSON của từng phần tử cho DocumentController.
5. DocumentController trả file đã được đóng gói cho SettingsPanel.
6. SettingsPanel kích hoạt tải xuống, file được lưu xuống thiết bị người dùng.



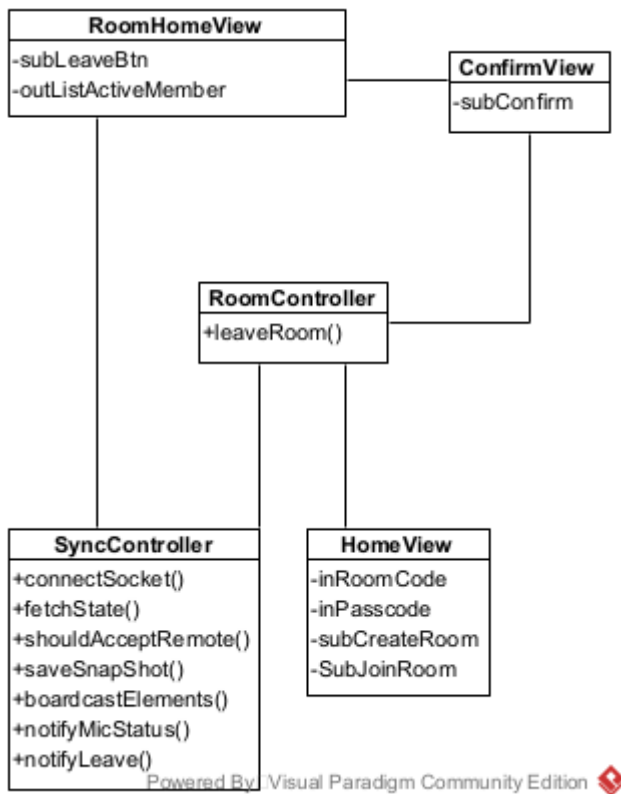
Hình 3.18:

Biểu đồ tuần tự — chức năng Xuất file bảng trắng {#**hình-3.18:-biểu-đồ-tuần-tự—chức-năng-xuất-file-bảng-trắng**}

## i. Chức năng rời phòng {#i.-chức-năng-rời-phòng}

### Phân tích tĩnh — các lớp và thành phần cần thiết:

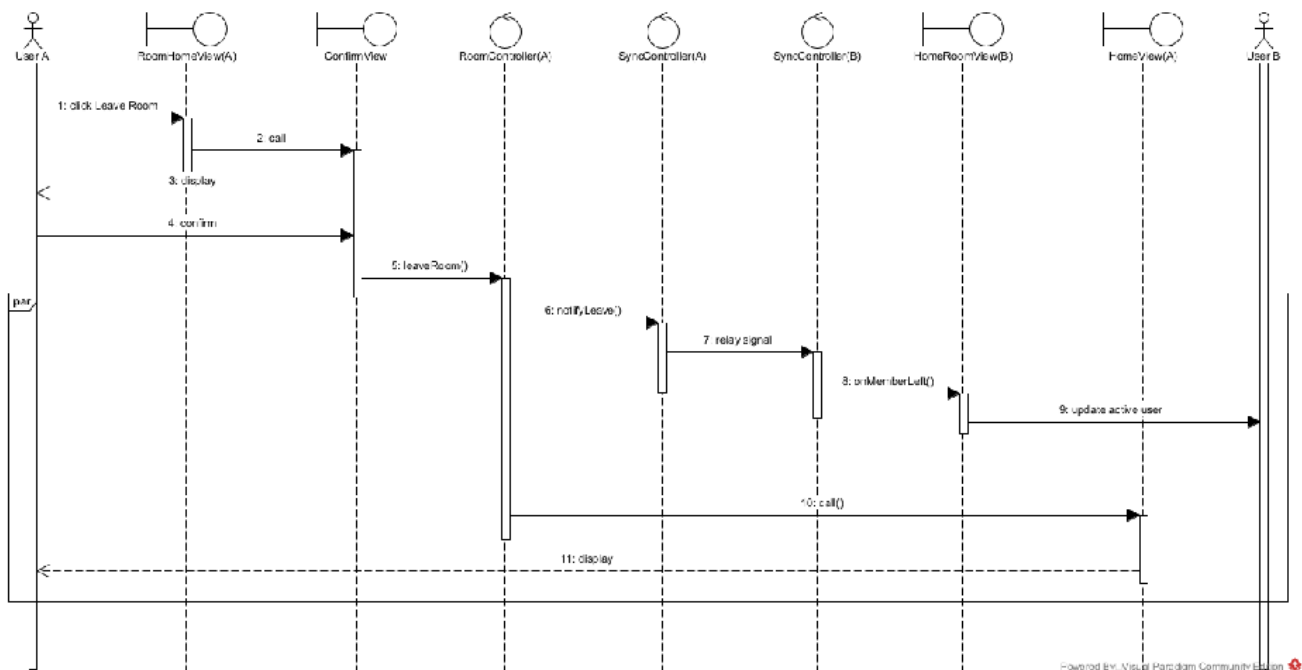
- NSD thao tác trên giao diện phòng → lớp **RoomPage** (đã có). Cần bổ sung chức năng `showLeaveConfirm()` để hiển thị hộp thoại xác nhận rời phòng, và `onMemberLeft(member)` để cập nhật giao diện khi một thành viên khác rời phòng.
- Hệ thống cần xử lý logic rời phòng → lớp điều khiển **RoomController** (đã có). Cần bổ sung chức năng `leaveRoom()` để điều phối toàn bộ quy trình rời phòng.
- Cần thông báo cho các thành viên khác biết có người rời phòng → lớp **SyncController** (đã có). Phát tín hiệu rời phòng qua sự kiện `Socket.IO leave_room`; server sẽ broadcast `user_left` tới các thành viên còn lại.
- Sau khi rời phòng, NSD được chuyển trở về trang chủ → lớp **LandingPage** (đã có).



Hình 3.19: Biểu đồ các lớp phân tích — chức năng Rời phòng

### Phân tích động — biểu đồ tuần tự:

1. Người dùng A click nút "Rời phòng" trên RoomPage.
2. RoomPage hiển thị hộp thoại xác nhận rời phòng. (đã sửa: xóa ConfirmView — component không tồn tại trong codebase)
3. Hộp thoại hiển thị cho người dùng.
4. Người dùng A xác nhận rời phòng.
5. Hộp thoại gọi RoomController.leaveRoom().
6. RoomController thực hiện đồng thời hai việc:
  1. **Thông báo:** RoomController phát sự kiện Socket.IO leave\_room; server broadcast user\_left tới các thành viên.
  2. **Điều hướng:** RoomController chuyển Người dùng A trở về LandingPage
    1. LandingPage hiển thị trang chủ cho người dùng.
7. SyncController (A) chuyển tiếp tín hiệu qua mạng tới SyncController (Máy B).
8. SyncController (B) gọi RoomPage(B).onMemberLeft(member) để cập nhật danh sách thành viên cho Người dùng B.



### 3.1.4. Thiết kế chi tiết hệ thống {#3.1.4.-thiết-kế-chi-tiết-hệ-thống}

Chuyển đổi thiết kế từ Hướng đối tượng sang Hướng chức năng: Mặc dù ở pha Phân tích hệ thống sử dụng tư duy Hướng đối tượng (OOP) thông qua biểu đồ UML để làm rõ các nghiệp vụ (Mô hình Boundary - Control - Entity), tuy nhiên khi bước vào pha Thiết kế và Cài đặt, hệ thống EvoDraw áp dụng mô hình thiết kế hỗn hợp để tối ưu hóa cho công nghệ thực tế. Cụ thể, hệ thống giữ nguyên cấu trúc OOP ở tầng Dữ liệu (Entity/Model) nhưng chuyển đổi sang tư duy Hướng chức năng (Functional Programming) ở tầng Điều khiển (Controller) và Giao diện (View). Quyết định này xuất phát từ đặc thù của công nghệ:

- **Tối ưu phía Client (React):** Việc sử dụng Functional Components và Custom Hooks thay cho Class Components (OOP truyền thống) là tiêu chuẩn của React. Nó giúp chia sẻ logic (như đồng bộ Socket) dễ dàng giữa các module, tránh lỗi rò rỉ bộ nhớ và khắc phục sự phức tạp của việc quản lý trạng thái qua đối tượng.
- **Tối ưu phía Server (Express):** Backend Node.js/Express bản chất xử lý các luồng HTTP Request/Response phi trạng thái. Việc cài đặt các API Controller dưới dạng các hàm chức năng độc lập thay vì khởi tạo các Class lớn giúp giảm thiểu độ trễ, tiết kiệm tài nguyên bộ nhớ máy chủ.

Dựa trên nguyên tắc trên, pha thiết kế tiến hành chuyển hóa các khái niệm trừu tượng sang các thành phần kỹ thuật tương ứng với kiến trúc **React (Frontend)** và **Express (Backend)** của hệ thống:

- Tầng **View**: Được xây dựng bằng các thành phần giao diện động (React Components).
- Tầng **Controller**: Được cài đặt thông qua các bộ điều khiển logic hiển thị ở Client (React Hooks) và các bộ xử lý API ở Server (Express Controllers).
- Tầng **Entity/Model**: Sử dụng Mongoose Schemas để lưu trữ cơ sở dữ liệu trên MongoDB và cấu trúc fabric.Object để biểu diễn đối tượng vẽ.

#### a. Hoàn thiện cấu trúc các thành phần thiết kế {#a.-hoàn-thiện-cấu-trúc-các-thành-phần-thiết-kế}

Hệ thống được thiết kế theo 3 tầng rõ rệt:

- **Tầng Thực thể (Model/Entity):** Bao gồm các lớp đại diện cho cấu trúc dữ liệu lưu trữ.
  - **Room:** code: String, passcode: String, status: String, appState: Object, roomVersion: Number, elements: Array. Tương ứng với createRoomService() và updateRoomService() trong room.service.js.
  - **File:** fileId: String, roomId: String, mimeType: String, dataURL: String, size: Number. Quản lý siêu dữ liệu (metadata) của ảnh.
  - **CanvasElement:** Đại diện cho các đối tượng vẽ trên bảng trắng (áp dụng cấu trúc nội tại của Fabric.js). Có phương thức toJSON() (nội tại của Fabric.js), được gọi tập trung bởi serializeCanvas() trong canvasSerializer.js.
- **Tầng Điều khiển (Controller):** Quản lý luồng tương tác, giao tiếp mạng và xử lý nghiệp vụ.
  - **SyncController:** Quản lý luồng đồng bộ thời gian thực qua WebSockets (Socket.IO). Có các phương thức broadcastOperation(), applyRemoteOperation().
  - **RoomController:** Quản lý vòng đời phòng họp. Có các phương thức API: createRoom(), joinRoom(), leaveRoom().
  - **MediaController:** Xử lý việc tải lên tệp tin nhị phân. Cung cấp hàm uploadFile() để đẩy ảnh lên bộ nhớ đám mây (Firebase Storage).
  - **StreamController:** Quản lý luồng chia sẻ màn hình và đàm thoại âm thanh qua WebRTC (LiveKit). Có các hàm: toggleVoice(), startSharing(), stopSharing(). (đã sửa: toggleMicrophone → toggleVoice, startScreenShare → startSharing, stopScreenShare → stopSharing)
  - **ChatController:** Quản lý luồng gửi và nhận tin nhắn trong phòng họp. Có phương thức sendMessage().
  - **DocumentController:** Xử lý tuần tự hóa và giải tuần tự hóa dữ liệu bảng trắng. Có hai phương thức: importFile() để đọc file JSON và khôi phục bảng trắng, exportFile() để đóng gói dữ liệu và kích hoạt tải xuống.
- **Tầng Giao diện (View):** Hiển thị và thu thập tương tác của người dùng.
  - **LandingPage:** Hiển thị giao diện trang chủ, biểu mẫu tạo và tham gia phòng.
  - **RoomPage:** Thành phần cha quản lý bố cục tổng thể của phòng họp.
  - **Canvas:** Khu vực vẽ chính, bắt các sự kiện tương tác chuột và cảm ứng.
  - **Toolbar:** Thanh công cụ, cho phép chọn màu sắc, kích thước và loại nét vẽ.
  - **ChatPanel:** Khung giao diện hiển thị và nhập tin nhắn.
  - **SettingsPanel:** Bảng điều khiển các cấu hình hệ thống (như thiết bị âm thanh) và chức năng Nhập/Xuất file.

## b. Thiết kế chi tiết các mô-đun (Dạng bảng) {#b.-thiết-kế-chi-tiết-các-mô-đun-(dạng-bảng)}

Dưới đây là bảng mô tả thiết kế chi tiết (Tabular design) cho các thao tác xử lý nghiệp vụ chính của hệ thống, chỉ rõ đầu vào, đầu ra của các mô-đun:

Bảng 3.1: Mô-đun SyncController.broadcastOperation {#bảng-3.1:-mô-đun-synccontroller.broadcastoperation}

| Đặc tả | Nội dung |
|--------|----------|
|--------|----------|



| Đặc tả                     | Nội dung                                                                                                                                                               |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Bắt các sự kiện thay đổi trên bảng trắng (như thêm/sửa/xóa nét vẽ), đóng gói thành các thao tác và phát tin tới các thiết bị khác trong cùng phòng họp qua WebSockets. |
| <b>Đầu vào</b>             | op (Đối tượng thao tác chứa hành động và nội dung phần tử bị thay đổi)                                                                                                 |
| <b>Đầu ra</b>              | Không (Thực thi bất đồng bộ)                                                                                                                                           |
| <b>Các mô-đun được gọi</b> | Kết nối mạng qua Socket.IO                                                                                                                                             |

Bảng 3.2: Mô-đun *SyncController.applyRemoteOperation* {#bảng-3.2:-mô-đun-synccontroller.applyremoteoperation}

| Đặc tả                     | Nội dung                                                                                                                                                                                                     |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Lắng nghe tín hiệu cập nhật từ máy khách khác. Áp dụng thuật toán giải quyết xung đột (cơ chế Last Write Wins - LWW) và cập nhật trực tiếp thành phần tương ứng lên bảng trắng mà không phải vẽ lại toàn bộ. |
| <b>Đầu vào</b>             | op (Đối tượng thao tác nhận được từ Server)                                                                                                                                                                  |
| <b>Đầu ra</b>              | Thay đổi giao diện bảng trắng                                                                                                                                                                                |
| <b>Các mô-đun được gọi</b> | Hàm cập nhật DOM nội tại của Fabric.js                                                                                                                                                                       |

Bảng 3.3: Mô-đun *MediaController.uploadFile* {#bảng-3.3:-mô-đun-mediacontroller.uploadfile}

| Đặc tả                     | Nội dung                                                                                                                                                                                                            |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Tiếp nhận tệp tin hình ảnh từ máy khách. Luân chuyển tệp tải lên dịch vụ bộ nhớ đám mây (Firebase Storage), sau đó lưu trữ các tham số metadata liên đới vào cơ sở dữ liệu trước khi trả về URL truy cập công khai. |
| <b>Đầu vào</b>             | Dữ liệu hình ảnh (Binary Buffer)                                                                                                                                                                                    |
| <b>Đầu ra</b>              | Chuỗi JSON chứa URL công khai của tệp                                                                                                                                                                               |
| <b>Các mô-đun được gọi</b> | SDK lưu trữ đám mây, Tầng thao tác CSDL (File.create())                                                                                                                                                             |

Bảng 3.4: Mô-đun `RoomController.createRoom` {#bảng-3.4:-mô-đun-roomcontroller.createRoom}

| Đặc tả                     | Nội dung                                                                                                                                                                             |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Chịu trách nhiệm khởi tạo một phiên làm việc mới. Hệ thống sẽ sinh mã phòng ngẫu nhiên và mã hóa một chiều mật khẩu truy cập (passcode) trước khi lưu trữ bản ghi vào cơ sở dữ liệu. |
| <b>Đầu vào</b>             | Mã PIN (passcode) do người quản trị thiết lập                                                                                                                                        |
| <b>Đầu ra</b>              | Trạng thái phòng họp, mã phòng và passcode đã mã hóa                                                                                                                                 |
| <b>Các mô-đun được gọi</b> | Thư viện băm bảo mật (bcrypt), <code>createRoomService()</code> trong <code>room.service.js</code>                                                                                   |

Bảng 3.5: Mô-đun `RoomController.joinRoom` {#bảng-3.5:-mô-đun-roomcontroller.joinroom}

| Đặc tả                     | Nội dung                                                                                                                                                                   |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Xác thực thông tin đăng nhập phòng. Tra cứu phòng theo mã, so khớp mã PIN với bản ghi đã băm trong CSDL, gia hạn thời gian tồn tại của phòng và mở kết nối thời gian thực. |
| <b>Đầu vào</b>             | Mã phòng (code) và mã PIN (passcode)                                                                                                                                       |
| <b>Đầu ra</b>              | Trạng thái phòng họp và dữ liệu bảng trắng hiện tại (snapshot)                                                                                                             |
| <b>Các mô-đun được gọi</b> | <code>getRoom({ code })</code> ( <code>room.service.js</code> ), <code>bcrypt.compare()</code> , <code>SyncController</code>                                               |

Bảng 3.6: Mô-đun `RoomController.leaveRoom` {#bảng-3.6:-mô-đun-roomcontroller.leaveroom}

| Đặc tả                     | Nội dung                                                                                                                         |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Điều phối quy trình rời phòng. Phát tín hiệu thông báo tới các thành viên còn lại và chuyển hướng người dùng về trang chủ.       |
| <b>Đầu vào</b>             | Không                                                                                                                            |
| <b>Đầu ra</b>              | Điều hướng về trang chủ                                                                                                          |
| <b>Các mô-đun được gọi</b> | Sự kiện <code>Socket.IO leave_room</code> → server broadcast <code>user_left</code> tới các thành viên, <code>LandingPage</code> |

Bảng 3.7: Mô-đun `StreamController.startSharing` (đã sửa: `startScreenShare` → `startSharing`) {#bảng-3.7:-mô-đun-streamcontroller.startsharing}

| Đặc tả | Nội dung |
|--------|----------|
|--------|----------|

| Đặc tả                     | Nội dung                                                                                                                                                                           |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Yêu cầu thiết bị cấp quyền truy cập màn hình, lấy luồng video và đẩy lên máy chủ SFU (MediaServer). Đồng thời phát tín hiệu báo các thành viên khác biết có người bắt đầu chia sẻ. |
| <b>Đầu vào</b>             | Không                                                                                                                                                                              |
| <b>Đầu ra</b>              | Luồng video màn hình được hiển thị trên bảng trắng của các thành viên                                                                                                              |
| <b>Các mô-đun được gọi</b> | MediaServer (LiveKit SFU), SyncController                                                                                                                                          |

**Bảng 3.8: Mô-đun StreamController.toggleVoice (đã sửa: toggleMicrophone → toggleVoice) {#bảng-3.8:-mô-đun-streamcontroller.togglevoice}**

| Đặc tả                     | Nội dung                                                                                                                                                                              |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Bật hoặc tắt micro của người dùng. Khi bật, yêu cầu cấp quyền micro, thu thập luồng âm thanh và đẩy lên MediaServer. Phát tín hiệu cập nhật trạng thái micro cho các thành viên khác. |
| <b>Đầu vào</b>             | Không                                                                                                                                                                                 |
| <b>Đầu ra</b>              | Trạng thái micro được cập nhật trên giao diện của tất cả thành viên                                                                                                                   |
| <b>Các mô-đun được gọi</b> | MediaServer (LiveKit SFU), SyncController                                                                                                                                             |

**Bảng 3.9: Mô-đun ChatController.sendMessage {#bảng-3.9:-mô-đun-chatcontroller.sendmessage}**

| Đặc tả                     | Nội dung                                                                                                                                                                       |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Khởi tạo đối tượng tin nhắn (chứa người gửi, nội dung, dấu thời gian), hiển thị tin nhắn tức thì trên màn hình người gửi và phát tin nhắn tới các thành viên khác trong phòng. |
| <b>Đầu vào</b>             | Nội dung tin nhắn (chuỗi văn bản)                                                                                                                                              |
| <b>Đầu ra</b>              | Tin nhắn được hiển thị trên ChatPanel của tất cả thành viên                                                                                                                    |
| <b>Các mô-đun được gọi</b> | ChatMessage (Entity), SyncController, ChatPanel                                                                                                                                |

**Bảng 3.10: Mô-đun DocumentController.importFile {#bảng-3.10:-mô-đun-documentcontroller.importfile}**

| Đặc tả | Nội dung |
|--------|----------|
|--------|----------|

| Đặc tả                     | Nội dung                                                                                                                                                               |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Đọc file JSON từ thiết bị người dùng, phân tích cú pháp và tái tạo các phần tử vẽ trên bảng trắng. Sau đó đồng bộ các phần tử mới tới các thành viên khác trong phòng. |
| <b>Đầu vào</b>             | Tệp tin JSON chứa dữ liệu các phần tử vẽ                                                                                                                               |
| <b>Đầu ra</b>              | Bảng trắng được khôi phục với các phần tử từ file                                                                                                                      |
| <b>Các mô-đun được gọi</b> | loadCanvasSnapshot() (canvasSerializer.js), SyncController.broadcastOperation()                                                                                        |

Bảng 3.11: Mô-đun *DocumentController.exportFile* {#bảng-3.11:-mô-đun-documentcontroller.exportfile}

| Đặc tả                     | Nội dung                                                                                                                                                                                 |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mô tả nghiệp vụ</b>     | Duyệt qua tất cả các phần tử vẽ hiện có trên bảng trắng, gọi hàm tuần tự hóa trên từng phần tử để thu thập dữ liệu, đóng gói thành file JSON và kích hoạt tải xuống thiết bị người dùng. |
| <b>Đầu vào</b>             | Danh sách các phần tử vẽ hiện có trên bảng trắng                                                                                                                                         |
| <b>Đầu ra</b>              | Tệp tin JSON được tải xuống thiết bị                                                                                                                                                     |
| <b>Các mô-đun được gọi</b> | serializeCanvas() (canvasSerializer.js)                                                                                                                                                  |

## 3.2. Thiết kế cơ sở dữ liệu {#3.2.-thiết-kế-cơ-sở-dữ-liệu}

### 3.2.1. Mô hình dữ liệu (Document Model) {#3.2.1.-mô-hình-dữ-liệu-(document-model)}

Hệ thống sử dụng **MongoDB**, một cơ sở dữ liệu NoSQL hướng tài liệu. Thay vì sử dụng các bảng và khóa ngoại cứng nhắc như SQL, MongoDB cho phép lưu trữ dữ liệu dưới dạng các Document JSON linh hoạt. Điều này đặc biệt phù hợp với các phần tử canvas có cấu trúc phức tạp và không đồng nhất. Dữ liệu của EvoDraw được phân loại thành hai nhóm chính dựa trên tính chất lưu trữ:

1. **Dữ liệu bền vững:** bao gồm thông tin phòng (Room) cùng trạng thái canvas snapshot nhúng bên trong, và thông tin tham chiếu file ảnh (File).
2. **Dữ liệu tức thời:** Lưu trữ trực tiếp trên bộ nhớ RAM của Server hoặc chỉ truyền qua broadcast Socket.IO, bao gồm vị trí con trỏ chuột (Cursor) của các thành viên (~20 lần/giây) và tin nhắn chat trong phòng. Việc này giúp giảm tải cho cơ sở dữ liệu vì các dữ liệu này có tần suất thay đổi cao và không cần tra cứu lại sau phiên làm việc.

Cơ sở dữ liệu chỉ lưu trữ trạng thái canvas dưới dạng mảng JSON (elements[]) mà không kiểm tra hoặc đọc nội dung bên trong. Toàn bộ logic render, conflict resolution và reconciliation được xử lý phía client, server chỉ đóng vai trò trung chuyển và lưu trữ snapshot.

### 3.2.2 Thiết kế vật lý (Physical Design) {#3.2.2-thiết-kế-vật-lý-(physical-design)}

Hệ thống sử dụng Mongoose ODM trên MongoDB. Mongoose cho phép định nghĩa Schema chặt chẽ ở tầng ứng dụng, hỗ trợ sẵn các middleware (timestamps, hooks), validation, và đặc biệt cho phép khai báo TTL Index trực tiếp trong Schema - rất phù hợp với yêu cầu phòng tự hết hạn sau 24 giờ. Toàn bộ truy cập CSDL được đóng gói trong tầng model và tầng service

Collection **rooms** (Lưu trữ thông tin phòng họp và canvas snapshot):

- `_id`: Định danh nội bộ do MongoDB sinh tự động.
- `code`: Mã phòng gồm 6 ký tự viết hoa ngẫu nhiên dùng để chia sẻ.
- `passcode`: Mã PIN bảo mật đã được bấm để xác thực quyền tham gia.
- `roomVersion`: Phiên bản của canvas snapshot hiện tại.
- `elements`: Mảng JSON chứa toàn bộ các phần tử canvas.
- `appState`: Trạng thái giao diện chia sẻ được của phòng (màu nền canvas, theme, zoom mặc định...), được đồng bộ giữa các thành viên.
- `status`: Trạng thái phòng hiện tại.
- `createdAt`: Thời điểm tạo phòng.
- `updatedAt`: Thời điểm phòng có tương tác gần nhất.

```
mongoose.Schema( { code: { type: String, required: true, unique: true, uppercase: true, trim: true, }, passcode: { type: String, required: true, }, roomVersion: { type: Number, default: 0, }, // annotations in the room elements: { type: Array, default: [], }, // (theme, zoom...) appState: { type: Object, default: {}, }, status: { type: String, default: 'active', }, }, { timestamps: true, minimize: false, } );
```

---

Collection **files** (Lưu trữ metadata tham chiếu tới file nhị phân - ảnh/PDF - được lưu trên Firebase Storage):

- `_id`: Định danh nội bộ do MongoDB sinh tự động.
- `fileId`: Định danh logic của file.
- `roomId`: Tham chiếu tới mã phòng chứa file.
- `mimeType`: Loại MIME của file (VD: image/png, image/jpeg,...).
- `dataURL`: URL công khai của file trên Firebase Storage.
- `size`: Kích thước file tính bằng byte (giới hạn tối đa 10 MB mỗi file).
- `lastRetrieved`: Thời điểm truy cập gần nhất.
- `created`: Thời điểm tải lên.

```
mongoose.Schema( { fileId: { type: String, required: true, index: true, }, roomId: { type: String, required: true, index: true, }, // (image/png, application/json...) mimeType: { type: String, required: true, }, // (URL on cloud) dataURL: { type: String, required: true, }, size: { type: Number, required: true, }, lastRetrieved: { type: Date, default: Date.now, }, }, { timestamps: { createdAt: 'created', updatedAt: false }, } );
```

---

Lý do chọn hybrid storage (MongoDB + Firebase Storage): tránh lưu chuỗi Base64 trực tiếp trong snapshot JSON của phòng làm document phải lưu trữ dữ liệu lớn và dễ chạm giới hạn BSON 16 MB của MongoDB. Firebase Storage làm tốt vai trò lưu đối tượng nhị phân tĩnh và tận dụng CDN toàn cầu để giảm độ trễ khi client tải ảnh, đồng thời giảm tải băng thông cho server.

### 3.3 Thực nghiệm (nếu có) : {#3.3-thực-nghiem-(nếu-có)-:}

- Link github: <https://github.com/KhangDaoz/evodraw>
- Sản phẩm thực nghiệm: [evodraw-web.vercel.app](https://evodraw-web.vercel.app)

---

## KẾT LUẬN, KIẾN NGHỊ {#kết-luận,-kiến-nghị}

---

---

## TÀI LIỆU THAM KHẢO {#tài-liệu-tham-khảo}

---

<https://vtv.vn/kinh-te/mckinsey-mo-hinh-lam-viec-tu-xa-len-ngoi-20220419063543657.htm>

---

## PHỤ LỤC CÀI ĐẶT VÀ TRIỂN KHAI : {#phụ-lục-cài-đặt-và-triển-khai-:}

---

- + Thiết lập môi trường
- + Cài đặt triển khai hệ thống
- + Hình ảnh sản phẩm.

**File thiết kế tương tác** (Figma,...) — gắn link vào báo cáo

**File UML** — đưa lên GitHub

**Source code + file cài build** — đưa lên GitHub

**Slide thuyết trình**

**Video demo sản phẩm** kèm giới thiệu hoạt động